

Отчет по лабораторной работе №3

Модификации градиентного спуска

Выполнили:

Новиков Алексей Георгиевич, группа М3233

Душкин Александр Алексеевич, группа М3233

Санкт-Петербург

2026

Содержание

1	Постановка задачи	2
2	Используемые модели задач	2
2.1	Хорошо обусловленная квадратичная функция	2
2.2	Плохо обусловленная квадратичная функция	3
2.3	Сложные функции	3
3	Описание реализованных методов	4
3.1	Momentum	4
3.2	Метод Нестерова	5
3.3	AdaGrad	5
3.4	RMSProp	6
3.5	AdaDelta	6
3.6	Adam	7
4	Структура программной реализации	7
5	Результаты вычислительных экспериментов	8
5.1	Квадратичные функции: лучшие параметры	8
5.2	Чувствительность к параметрам	10
5.2.1	Хорошо обусловленная квадратичная функция	10
5.2.2	Плохо обусловленная квадратичная функция	14
5.3	Траектории на квадратичных функциях	22
5.4	Сложные функции: таблицы запусков	25
5.5	Сложные функции: траектории	28
5.5.1	Rosenbrock	29
5.5.2	Ackley	33
5.5.3	Himmelblau	37
6	Сводная таблица эксперимента	40
7	Общий вывод	40

1 Постановка задачи

Цель работы — реализовать и исследовать модификации градиентного спуска, в которых используются инерция, накопление статистики градиентов и адаптивный масштаб шага по координатам.

В работе реализованы:

1. Momentum;
2. Nesterov accelerated gradient;
3. AdaGrad;
4. RMSProp;
5. AdaDelta;
6. Adam.

Во всех экспериментах использовалась фиксированная точность

$$\varepsilon = 10^{-8},$$

а критерием остановки была норма градиента:

$$\|\nabla f(x_k)\| \leq \varepsilon.$$

Промежуточный вывод. Третья лабораторная продолжает вторую: вместо чистого шага по антиградиенту используются правила, которые помнят прошлые шаги или прошлые градиенты. Поэтому здесь особенно важны гиперпараметры. Они задают, насколько сильно метод опирается на текущий градиент, накопленную скорость и координатные масштабы.

2 Используемые модели задач

2.1 Хорошо обусловленная квадратичная функция

Первая тестовая функция:

$$f(x, y) = \frac{1}{2}(x^2 + y^2).$$

Минимум достигается в точке $(0, 0)$, значение минимума равно нулю. Матрица Гессе равна единичной, поэтому число обусловленности равно 1.

Промежуточный вывод. Эта функция удобна как базовый эталон: кривизна по обеим координатам одинакова, а антиградиент из любой точки направлен прямо к минимуму. Поэтому здесь почти не мешают геометрические эффекты вроде зигзагов, и различия между методами в основном связаны именно с выбором длины шага.

2.2 Плохо обусловленная квадратичная функция

Вторая квадратичная функция:

$$g(x, y) = \frac{1}{2}x^2 + 50y^2.$$

Ее градиент имеет вид

$$\nabla g(x, y) = (x, 100y).$$

Число обусловленности матрицы Гессе равно 100.

Промежуточный вывод. Плохая обусловленность приводит к вытянутым эллиптическим линиям уровня, потому что по координате y кривизна в сто раз больше, чем по координате x . Градиент указывает направление наискорейшего локального убывания, но не учитывает, что вдоль разных осей допустимы разные масштабы движения. Поэтому слишком большой общий шаг легко перескакивает через узкую долину, а слишком малый делает продвижение вдоль пологой координаты медленным.

2.3 Сложные функции

Для проверки поведения методов за пределами простых выпуклых квадратичных моделей использовались функции Розенброка, Экли и Химмельблау.

Функция Розенброка.

$$r(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

Глобальный минимум достигается в точке $(1, 1)$.

Функция Экли.

$$a(x, y) = -20 \exp \left(-0.2 \sqrt{\frac{x^2 + y^2}{2}} \right) - \exp \left(\frac{\cos(2\pi x) + \cos(2\pi y)}{2} \right) + e + 20.$$

Глобальный минимум находится в точке $(0, 0)$, однако поверхность содержит большое количество локальных особенностей.

Функция Химмельблау.

$$h(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2.$$

Функция имеет четыре глобальных минимума.

Промежуточный вывод. Эти функции добавляют разные сложности, которые хорошо видны именно у градиентных методов. У Розенброка анти-градиент часто направлен поперек узкой долины, поэтому появляются длинные серии корректирующих шагов. У Экли локальные колебания меняют направление градиента и могут увести метод в соседнюю область. У Химмельблау несколько минимумов, поэтому результат зависит еще и от стартовой точки.

3 Описание реализованных методов

3.1 Momentum

Метод Momentum добавляет инерционный член:

$$v_t = \beta v_{t-1} + \alpha \nabla f(x_t),$$

$$x_{t+1} = x_t - v_t.$$

Параметр β отвечает за сохранение предыдущего направления движения.

Промежуточный вывод. Инерция ускоряет движение вдоль устойчивого направления: новый шаг складывается не только из текущего градиента, но и из накопленной скорости. Если несколько градиентов подряд смотрят примерно в одну сторону, скорость растет и метод быстрее проходит пологие

участки. Но около минимума или в узкой долине это может мешать: накопленная скорость продолжает тянуть точку вперед, хотя текущий градиент уже требует торможения или поворота.

3.2 Метод Нестерова

В методе Нестерова градиент вычисляется в точке предпросмотра:

$$\begin{aligned}\tilde{x}_t &= x_t - \beta v_{t-1}, \\ v_t &= \beta v_{t-1} + \alpha \nabla f(\tilde{x}_t), \\ x_{t+1} &= x_t - v_t.\end{aligned}$$

Промежуточный вывод. Метод Нестерова пытается заранее оценить, куда приведет инерция, поэтому градиент вычисляется не в текущей точке, а в точке предпросмотра. Такая структура делает корректировку направления более ранней: метод видит последствия накопленной скорости до фактического обновления. За это он платит дополнительным вычислением градиента на каждой итерации, что видно в счетчиках вызовов.

3.3 AdaGrad

AdaGrad накапливает сумму квадратов градиентов:

$$\begin{aligned}s_t &= s_{t-1} + \nabla f(x_t)^2, \\ x_{t+1} &= x_t - \alpha \frac{\nabla f(x_t)}{\sqrt{s_t} + \varepsilon_{\text{num}}}.\end{aligned}$$

Промежуточный вывод. AdaGrad уменьшает эффективный шаг по координатам, где градиенты часто велики, потому что знаменатель содержит накопленную сумму квадратов градиента. Это полезно при разных масштабах координат: направление автоматически нормируется по истории. Однако накопитель только растет, поэтому на длинных детерминированных запусках эффективный шаг монотонно уменьшается и метод может почти остановиться до достижения требуемой точности.

3.4 RMSProp

RMSProp заменяет полную сумму квадратов экспоненциальным средним:

$$s_t = \rho s_{t-1} + (1 - \rho) \nabla f(x_t)^2,$$

$$x_{t+1} = x_t - \alpha \frac{\nabla f(x_t)}{\sqrt{s_t} + \varepsilon_{\text{num}}}.$$

Промежуточный вывод. Экспоненциальное сглаживание предотвращает бесконечный рост накопителя, потому что старые квадраты градиента постепенно забываются с коэффициентом ρ . В отличие от AdaGrad, RMSProp масштабирует шаг по недавней, а не по всей накопленной истории. Поэтому метод лучше адаптируется, когда масштаб градиента меняется по мере движения к минимуму.

3.5 AdaDelta

AdaDelta использует экспоненциальные средние для квадратов градиентов и обновлений:

$$E[g^2]_t = \rho E[g^2]_{t-1} + (1 - \rho) g_t^2,$$

$$\Delta x_t = - \frac{\sqrt{E[\Delta x^2]_{t-1} + \varepsilon_{\text{num}}}}{\sqrt{E[g^2]_t + \varepsilon_{\text{num}}}} g_t,$$

$$E[\Delta x^2]_t = \rho E[\Delta x^2]_{t-1} + (1 - \rho) \Delta x_t^2.$$

Промежуточный вывод. Метод не требует явного глобального шага, потому что масштаб обновления определяется отношением накопленного среднего квадратов прошлых шагов к среднему квадратов градиентов. Такая самонормировка делает AdaDelta устойчивым к выбору learning rate, но в рассматриваемых задачах приводит к консервативным обновлениям: если средний размер прошлых шагов мал, новые шаги также остаются малыми, и метод часто доходит до лимита итераций.

3.6 Adam

Adam объединяет Momentum и RMSProp:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Используется коррекция смещения:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Обновление:

$$x_{t+1} = x_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon_{\text{num}}}.$$

Промежуточный вывод. Adam выглядит самым сбалансированным из рассмотренных адаптивных методов: первый момент m_t сглаживает направление движения, а второй момент v_t нормирует шаг по координатам. Коррекция смещения нужна в начале оптимизации, когда накопители еще почти пустые и без поправки первые обновления были бы заметно искажены.

4 Структура программной реализации

Лабораторная работа построена поверх той же библиотеки `optimlib`, что и лабораторная №2. Ключевые элементы:

- `AdaptiveOptimizer` — базовый класс для методов с внутренним состоянием;
- `_reset_state()` — инициализация буферов перед каждым запуском;
- `_adaptive_direction(...)` — метод, в котором конкретный оптимизатор строит вектор обновления;
- `Momentum`, `Nesterov`, `AdaGrad`, `RMSProp`, `AdaDelta`, `Adam` — реализации методов;
- `OptimizationExperiment` — перебор функций, стартовых точек и сеток параметров;

- `plot_param_sensitivity` — построение line-графиков и heatmap чувствительности.

Сетки гиперпараметров задаются в `lab3/config.yaml`. Для Momentum и Nesterov перебираются α и β , для RMSProp — α и ρ , для AdaGrad — α , для AdaDelta — ρ , для Adam — α , β_1 и β_2 .

Промежуточный вывод. Архитектура оставляет общий цикл оптимизации в базовом классе. Новые методы различаются только правилом обновления и своими буферами состояния: скоростью, накопителем квадратов градиента или моментами Adam.

5 Результаты вычислительных экспериментов

Во всех экспериментах использовалась точность $\varepsilon = 10^{-8}$. Критерий сходимости:

$$\|\nabla f(x_k)\| \leq 10^{-8}.$$

В таблицах значение `converged` со значением `true` означает выполнение этого критерия. Для многоэкстремальных функций это не гарантирует попадание именно в глобальный минимум: метод может остановиться в другой стационарной области. Для Ackley около точки $(0, 0)$ дополнительно используется численная окрестность, потому что радиальный член с корнем имеет особенность в глобальном минимуме.

5.1 Квадратичные функции: лучшие параметры

Для квадратичных функций стартовая точка была фиксированной:

$$x_0 = (-2, -2).$$

Параметры перебирались так:

- Momentum и Nesterov: $\alpha \times \beta$;
- AdaGrad: α ;
- RMSProp: $\alpha \times \rho$;

- AdaDelta: ρ ;
- Adam: $\beta_1 \times \beta_2$ при фиксированном $\alpha = 10^{-2}$.

Таблица 1: Лучшие параметры на квадратичных функциях при $\varepsilon = 10^{-8}$

Функция	Метод	Лучший набор параметров	Итерации
f	Momentum	alpha=0.01, beta=0.85	182
f	Nesterov	alpha=0.01, beta=0.85	202
f	AdaGrad	alpha=0.1	1500
f	RMSProp	alpha=0.01, rho=0.85	238
f	AdaDelta	rho=0.5	1500
f	Adam	alpha=0.01, beta1=0.7, beta2=0.9	289
g	Momentum	alpha=0.01, beta=0.85	260
g	Nesterov	alpha=0.01, beta=0.85	202
g	AdaGrad	alpha=0.1	1500
g	RMSProp	alpha=0.01, rho=0.85	239
g	AdaDelta	rho=0.5	1500
g	Adam	alpha=0.01, beta1=0.7, beta2=0.95	362

Комментарий к таблице. На квадратичных функциях инерционные методы и RMSProp обычно выигрывают по числу итераций. Методы AdaGrad и AdaDelta часто доходят до лимита: это не ошибка запуска, а следствие осторожного уменьшения эффективного шага в выбранной сетке.

5.2 Чувствительность к параметрам

5.2.1 Хорошо обусловленная квадратичная функция

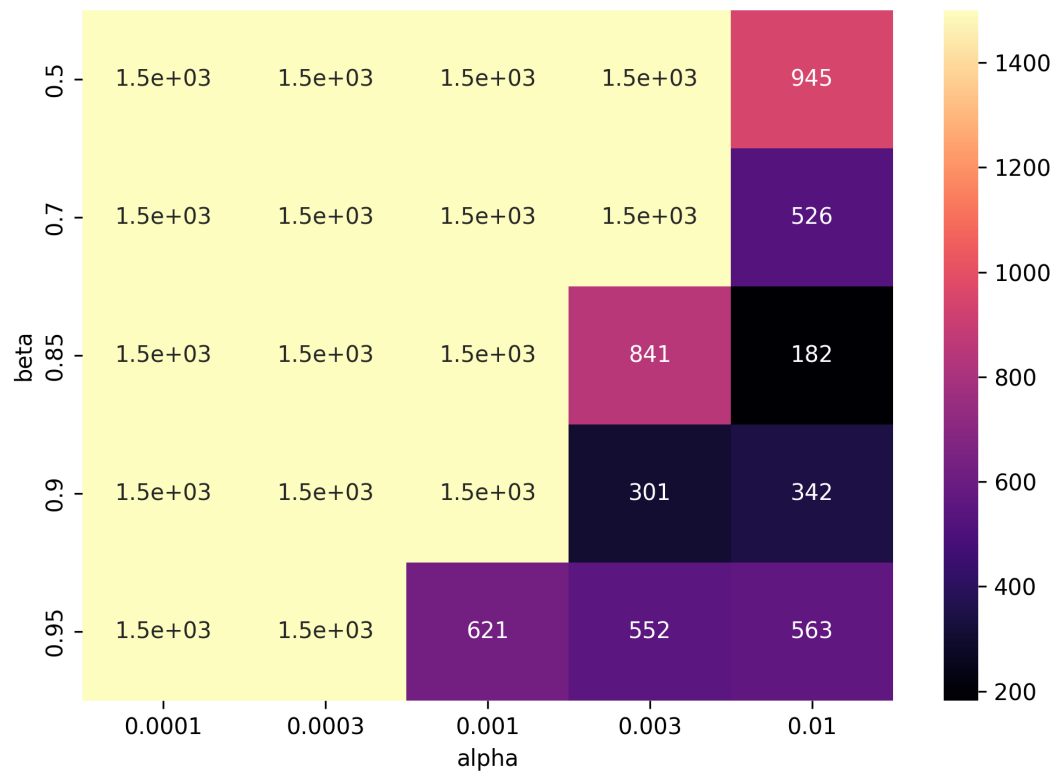


Рис. 1: Momentum: чувствительность к α и β на хорошо обусловленной функции

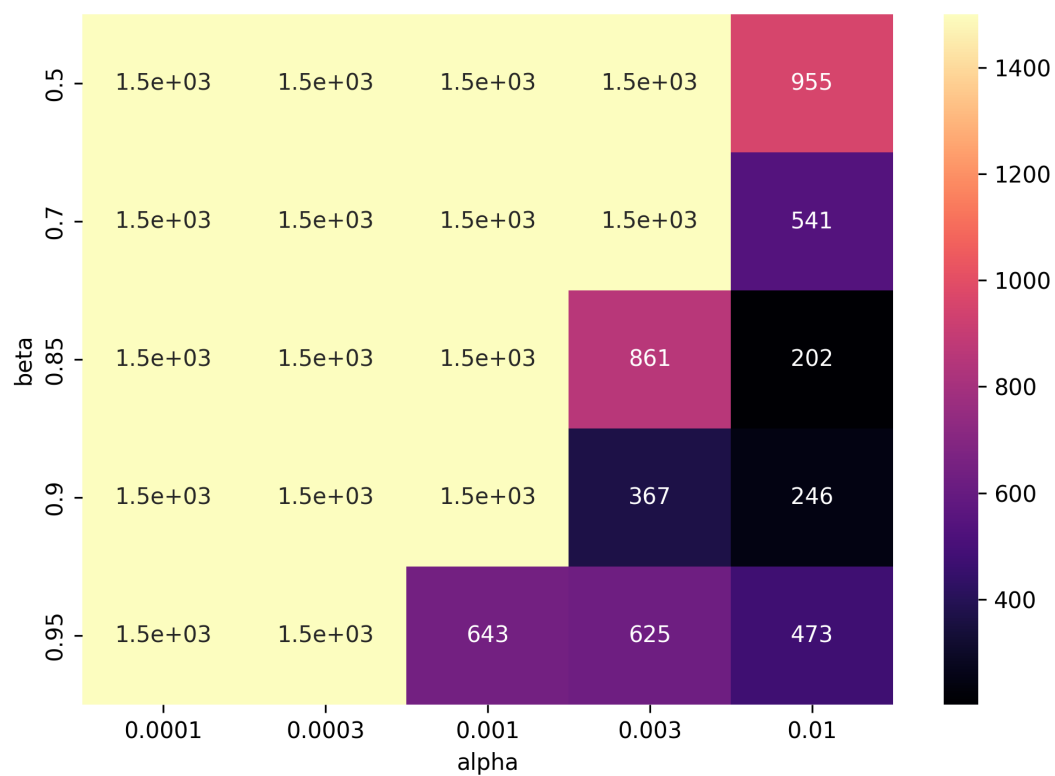


Рис. 2: Nesterov: чувствительность к α и β на хорошо обусловленной функции

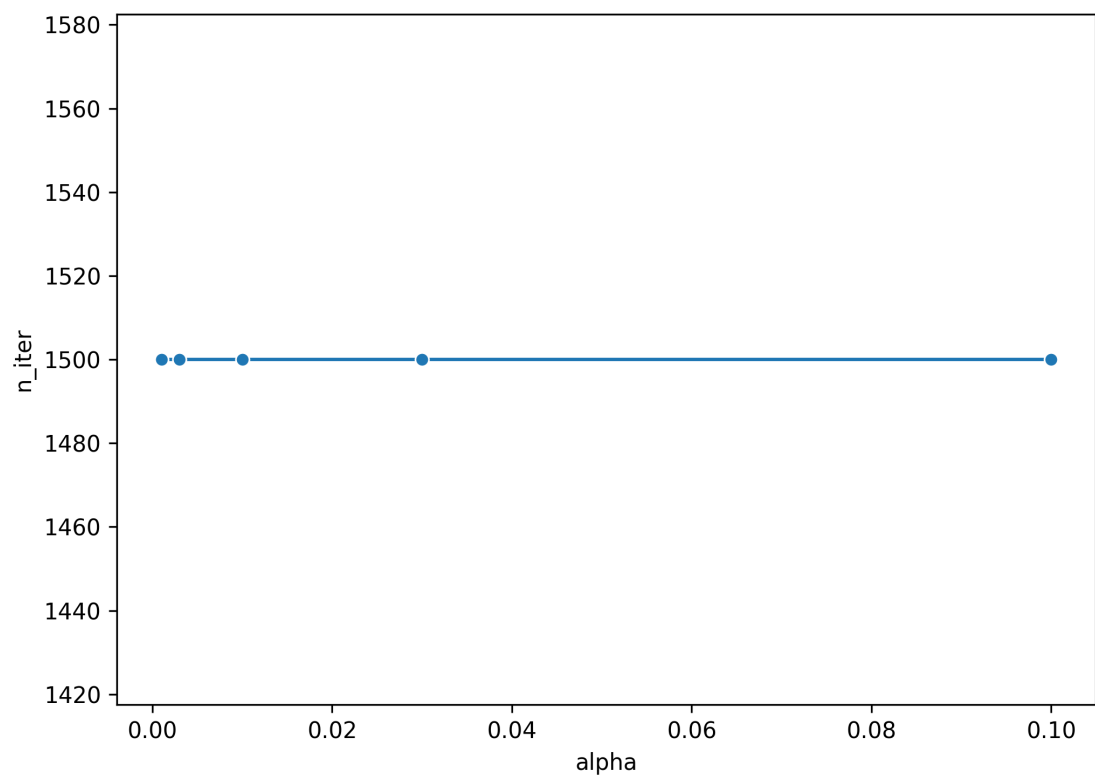


Рис. 3: AdaGrad: чувствительность к α на хорошо обусловленной функции

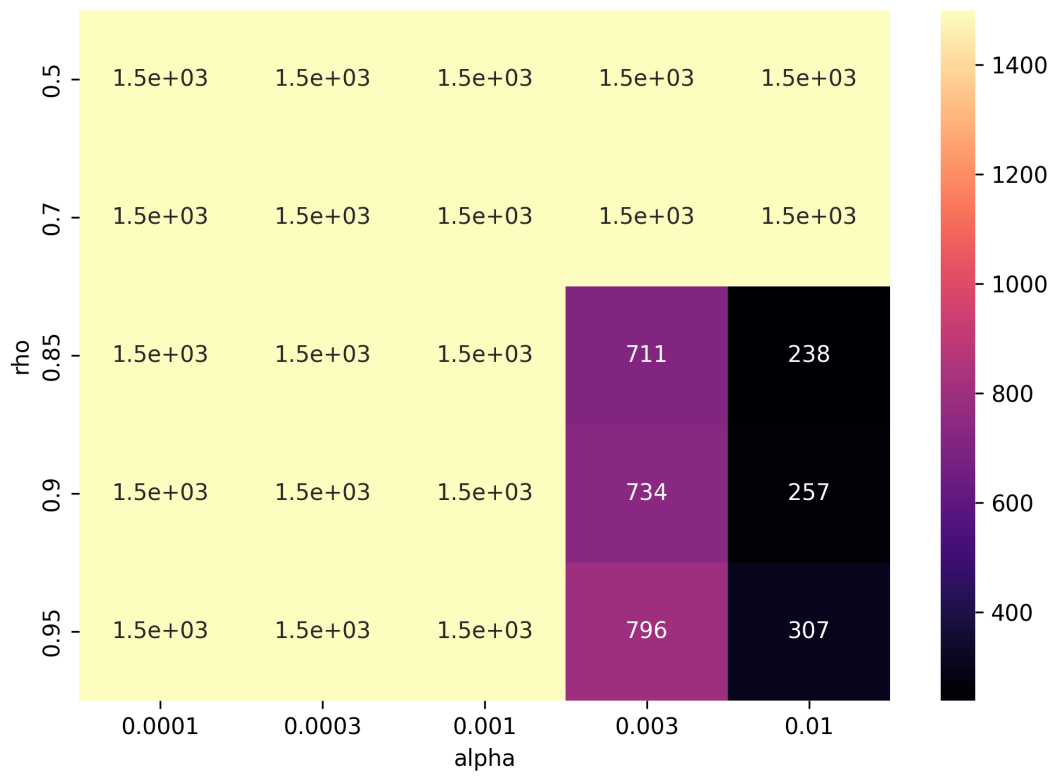


Рис. 4: RMSProp: чувствительность к α и ρ на хорошо обусловленной функции

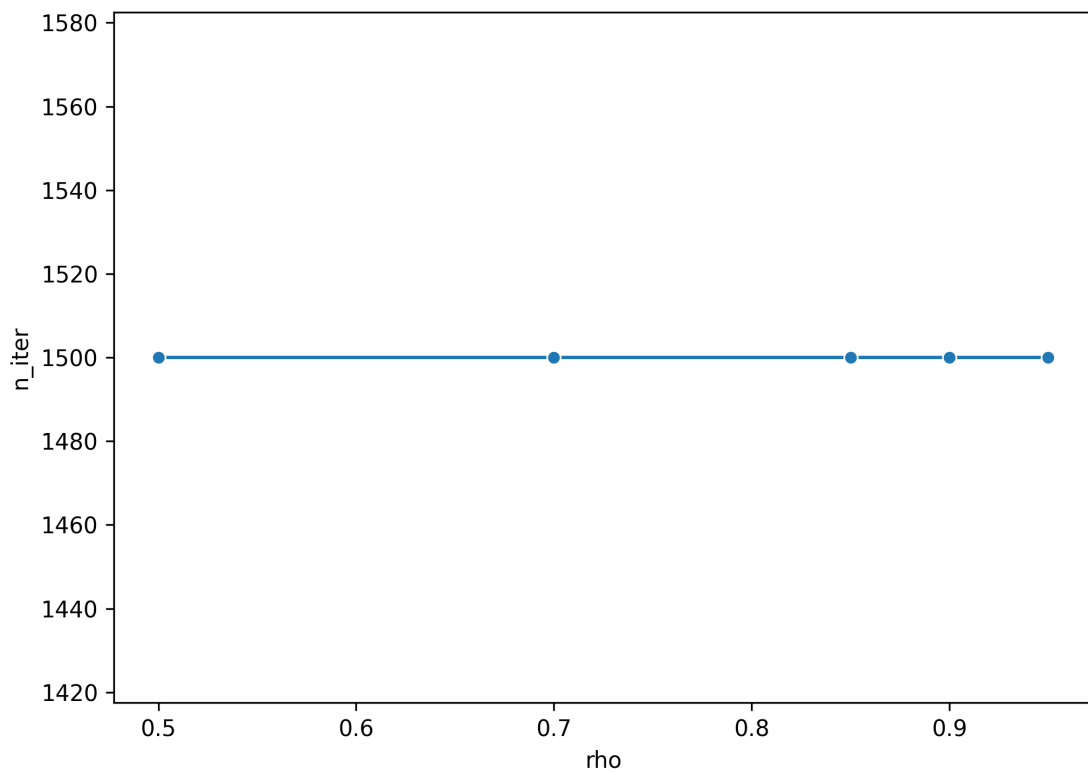


Рис. 5: AdaDelta: чувствительность к ρ на хорошо обусловленной функции

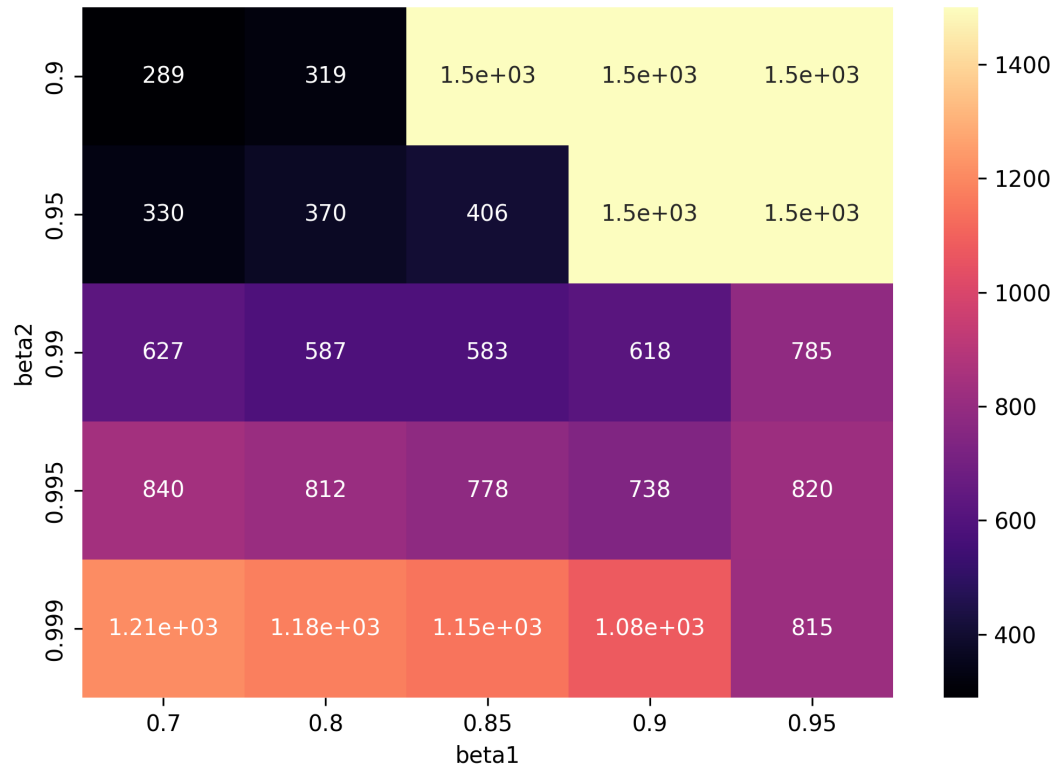


Рис. 6: Adam: чувствительность к β_1 и β_2 на хорошо обусловленной функции

Комментарий к графикам. На хорошо обусловленной функции удачная область параметров шире: кривизна по координатам одинакова. Поэтому главный вклад дают длина шага и инерция.

5.2.2 Плохо обусловленная квадратичная функция

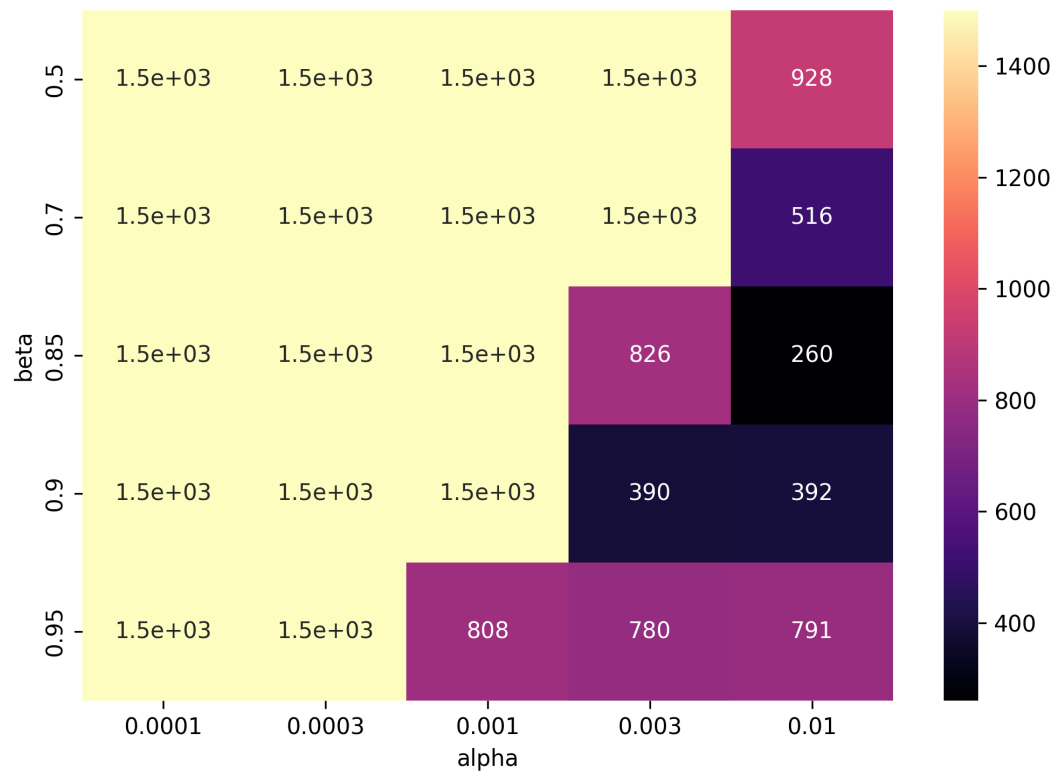


Рис. 7: Momentum: чувствительность к α и β на плохо обусловленной функции

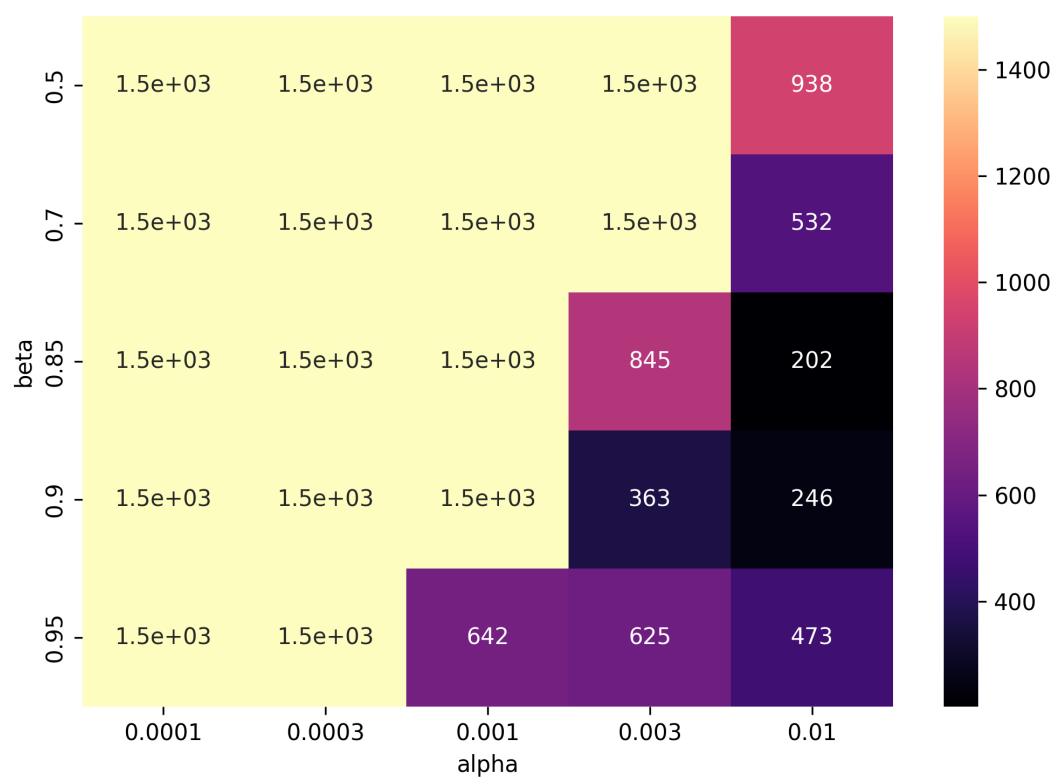


Рис. 8: Nesterov: чувствительность к α и β на плохо обусловленной функции

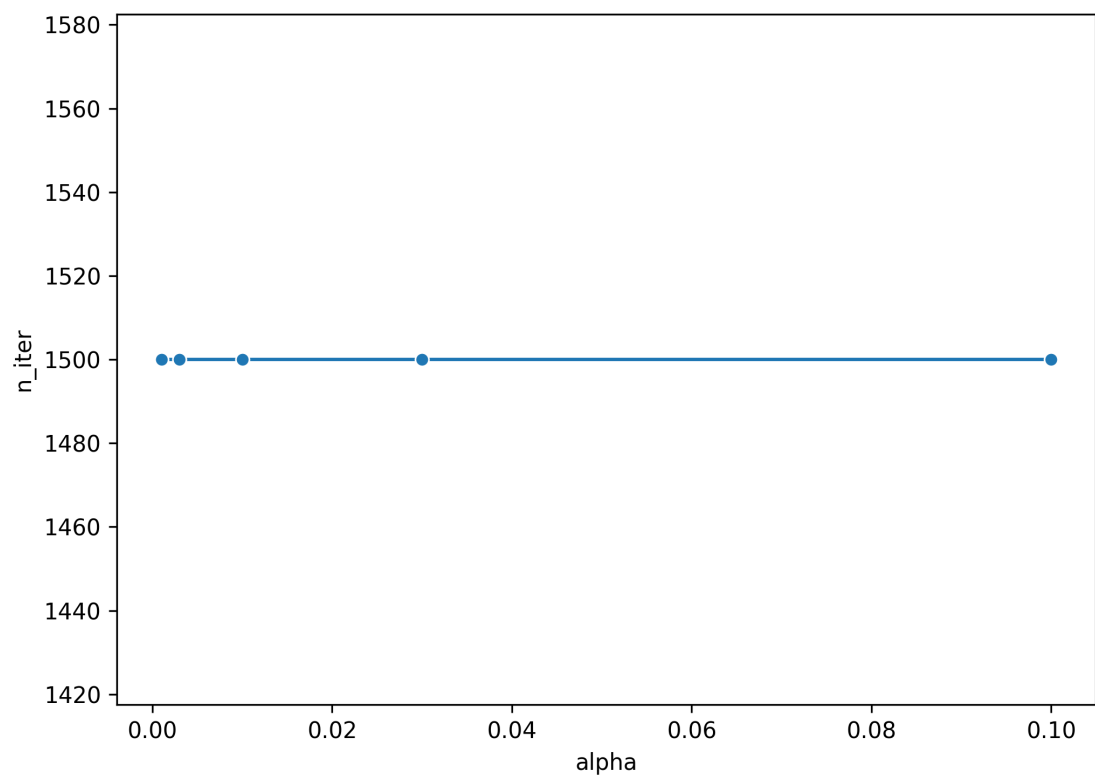


Рис. 9: AdaGrad: чувствительность к α на плохо обусловленной функции

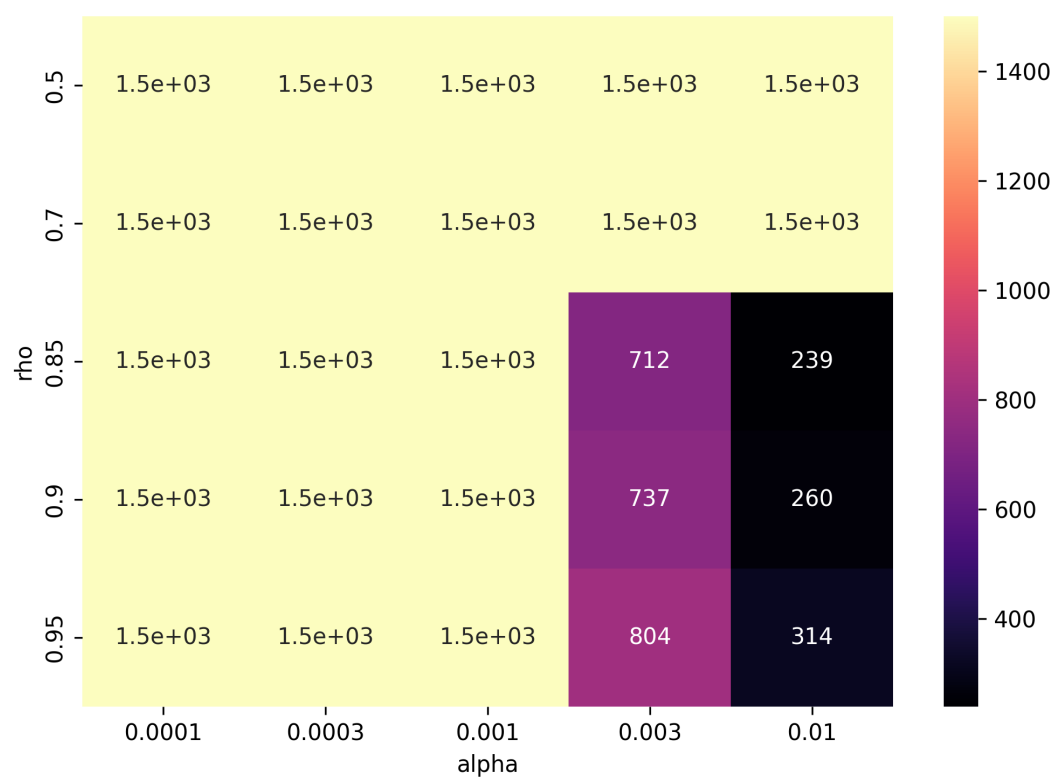


Рис. 10: RMSProp: чувствительность к α и ρ на плохо обусловленной функции

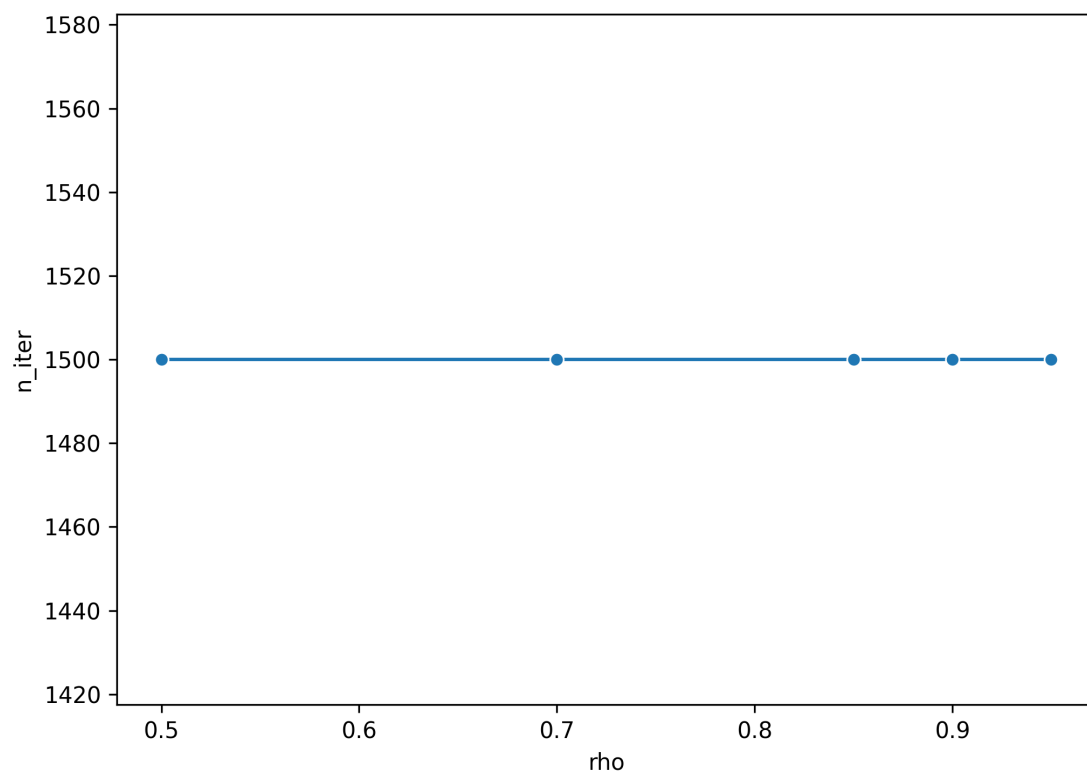


Рис. 11: AdaDelta: чувствительность к ρ на плохо обусловленной функции

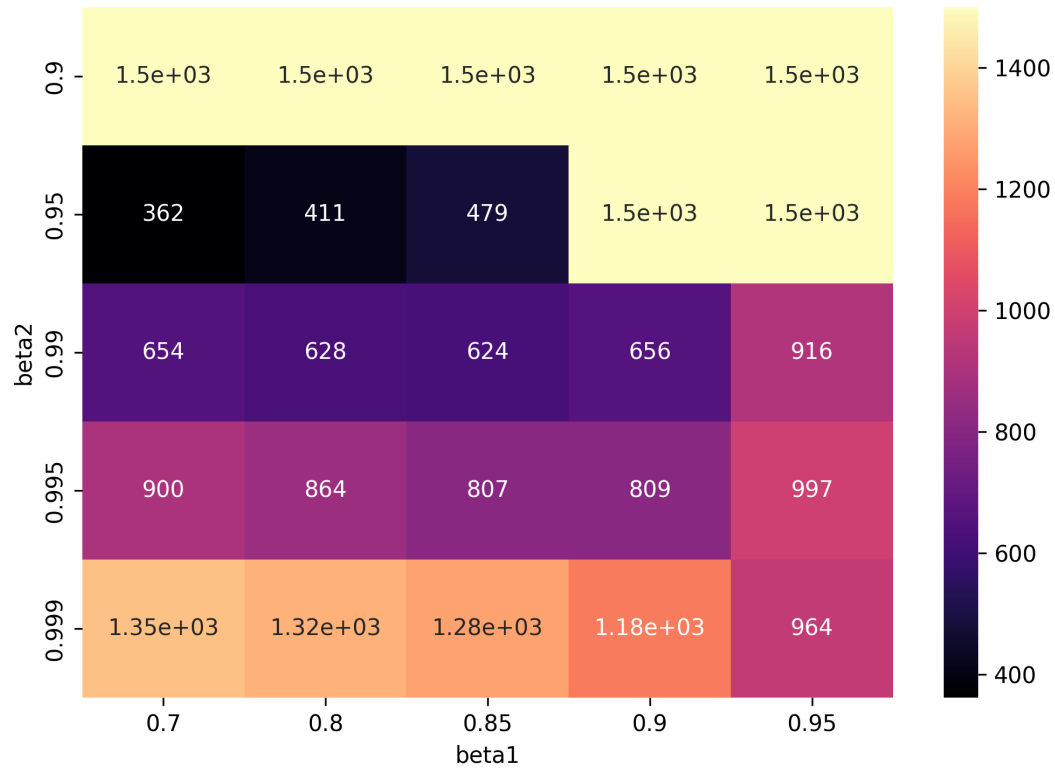


Рис. 12: Adam: чувствительность к β_1 и β_2 на плохо обусловленной функции

Комментарий к графикам. На плохо обусловленной функции удачная область параметров уже: слишком большой шаг или слишком сильная инерция быстро приводят к колебаниям поперек узкой долины. Адаптивные методы выигрывают за счет уменьшения шага по координате с большими недавними градиентами.

Примечание. Число со звездочкой означает, что метод дошел до лимита итераций без выполнения критерия по норме градиента.

Хорошо обусловленная квадратичная функция, $x_0 = (-2, -2)$

Таблица 2: Momentum: число итераций

beta / alpha	0.0001	0.0003	0.001	0.003	0.01
0.5	1500*	1500*	1500*	1500*	945
0.7	1500*	1500*	1500*	1500*	526
0.85	1500*	1500*	1500*	841	182
0.9	1500*	1500*	1500*	301	342
0.95	1500*	1500*	621	552	563

Таблица 3: Nesterov: число итераций

beta / alpha	0.0001	0.0003	0.001	0.003	0.01
0.5	1500*	1500*	1500*	1500*	955
0.7	1500*	1500*	1500*	1500*	541
0.85	1500*	1500*	1500*	861	202
0.9	1500*	1500*	1500*	367	246
0.95	1500*	1500*	643	625	473

Таблица 4: RMSProp: число итераций

rho / alpha	0.0001	0.0003	0.001	0.003	0.01
0.5	1500*	1500*	1500*	1500*	1500*
0.7	1500*	1500*	1500*	1500*	1500*
0.85	1500*	1500*	1500*	711	238
0.9	1500*	1500*	1500*	734	257
0.95	1500*	1500*	1500*	796	307

Таблица 5: Adam: число итераций

beta1 / beta2	0.9	0.95	0.99	0.995	0.999
0.7	289	330	627	840	1207
0.8	319	370	587	812	1178
0.85	1500*	406	583	778	1147
0.9	1500*	1500*	618	738	1075
0.95	1500*	1500*	785	820	815

Таблица 6: AdaGrad: число итераций

alpha	Итерации	Сошелся
0.001	1500	false
0.003	1500	false
0.01	1500	false
0.03	1500	false
0.1	1500	false

Таблица 7: AdaDelta: число итераций

rho	Итерации	Сошелся
0.5	1500	false
0.7	1500	false
0.85	1500	false
0.9	1500	false
0.95	1500	false

Плохо обусловленная квадратичная функция, $x_0 = (-2, -2)$

Таблица 8: Momentum: число итераций

beta / alpha	0.0001	0.0003	0.001	0.003	0.01
0.5	1500*	1500*	1500*	1500*	928
0.7	1500*	1500*	1500*	1500*	516
0.85	1500*	1500*	1500*	826	260
0.9	1500*	1500*	1500*	390	392
0.95	1500*	1500*	808	780	791

Таблица 9: Nesterov: число итераций

beta / alpha	0.0001	0.0003	0.001	0.003	0.01
0.5	1500*	1500*	1500*	1500*	938
0.7	1500*	1500*	1500*	1500*	532
0.85	1500*	1500*	1500*	845	202
0.9	1500*	1500*	1500*	363	246
0.95	1500*	1500*	642	625	473

Таблица 10: RMSProp: число итераций

rho / alpha	0.0001	0.0003	0.001	0.003	0.01
0.5	1500*	1500*	1500*	1500*	1500*
0.7	1500*	1500*	1500*	1500*	1500*
0.85	1500*	1500*	1500*	712	239
0.9	1500*	1500*	1500*	737	260
0.95	1500*	1500*	1500*	804	314

Таблица 11: Adam: число итераций

beta1 / beta2	0.9	0.95	0.99	0.995	0.999
0.7	1500*	362	654	900	1351
0.8	1500*	411	628	864	1316
0.85	1500*	479	624	807	1278
0.9	1500*	1500*	656	809	1185
0.95	1500*	1500*	916	997	964

Таблица 12: AdaGrad: число итераций

alpha	Итерации	Сошелся
0.001	1500	false
0.003	1500	false
0.01	1500	false
0.03	1500	false
0.1	1500	false

Таблица 13: AdaDelta: число итераций

rho	Итерации	Сошелся
0.5	1500	false
0.7	1500	false
0.85	1500	false
0.9	1500	false
0.95	1500	false

5.3 Трактории на квадратичных функциях

Для каждой квадратичной функции выбран один набор параметров на метод: если был сошедшийся запуск, выбирался минимальный по числу итераций; если метод не сходил в сетке, выбирался запуск с лучшим конечным значением функции. Чтобы легенды были читаемыми, методы разделены на инерционные и адаптивные.

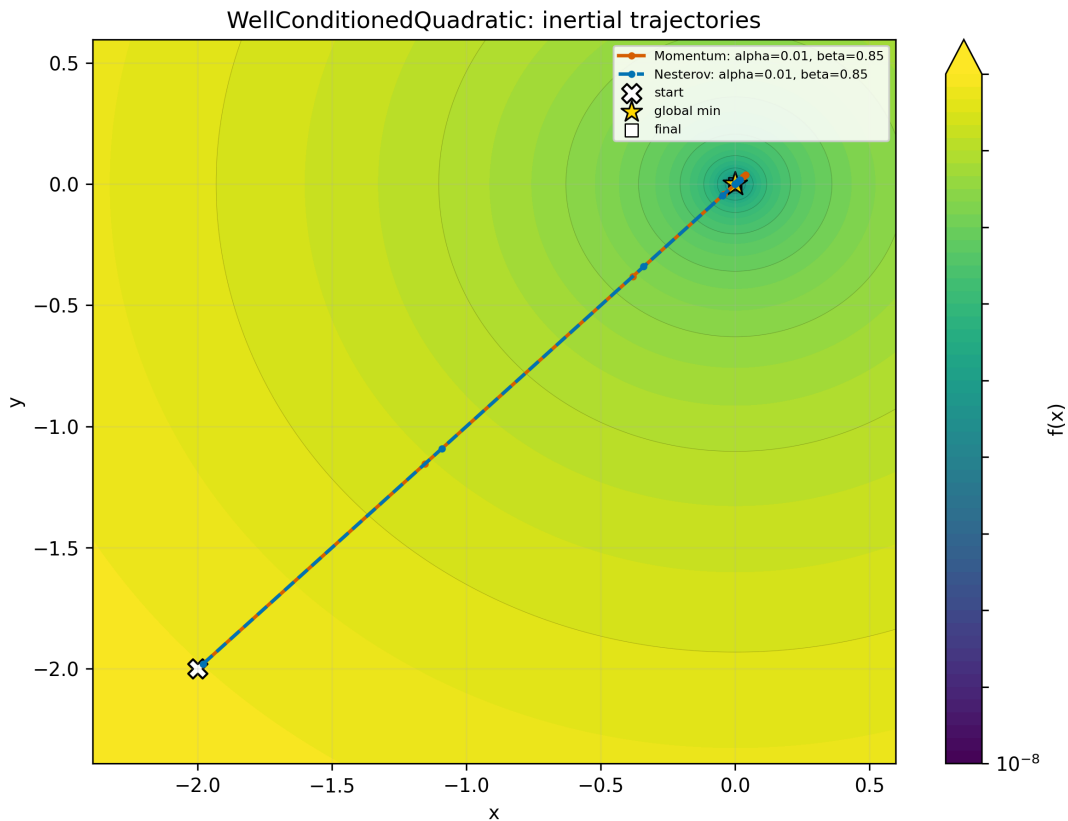


Рис. 13: Инерционные методы на хорошо обусловленной квадратичной функции

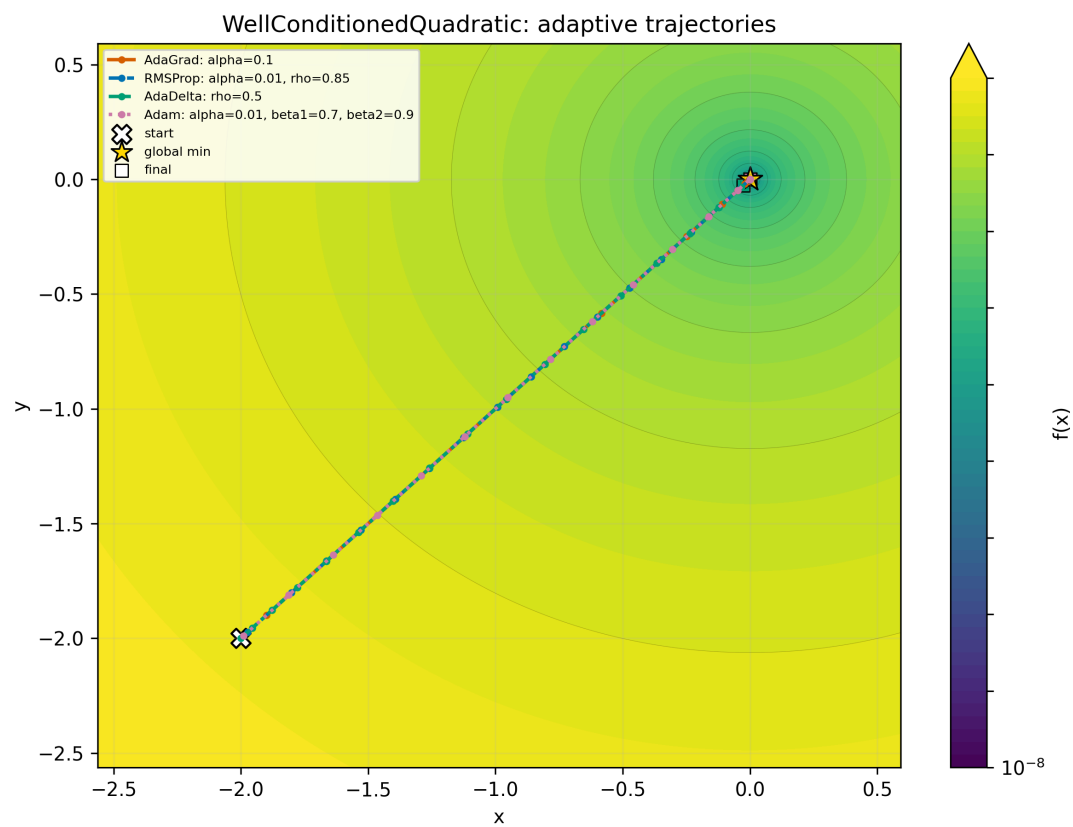


Рис. 14: Адаптивные методы на хорошо обусловленной квадратичной функции

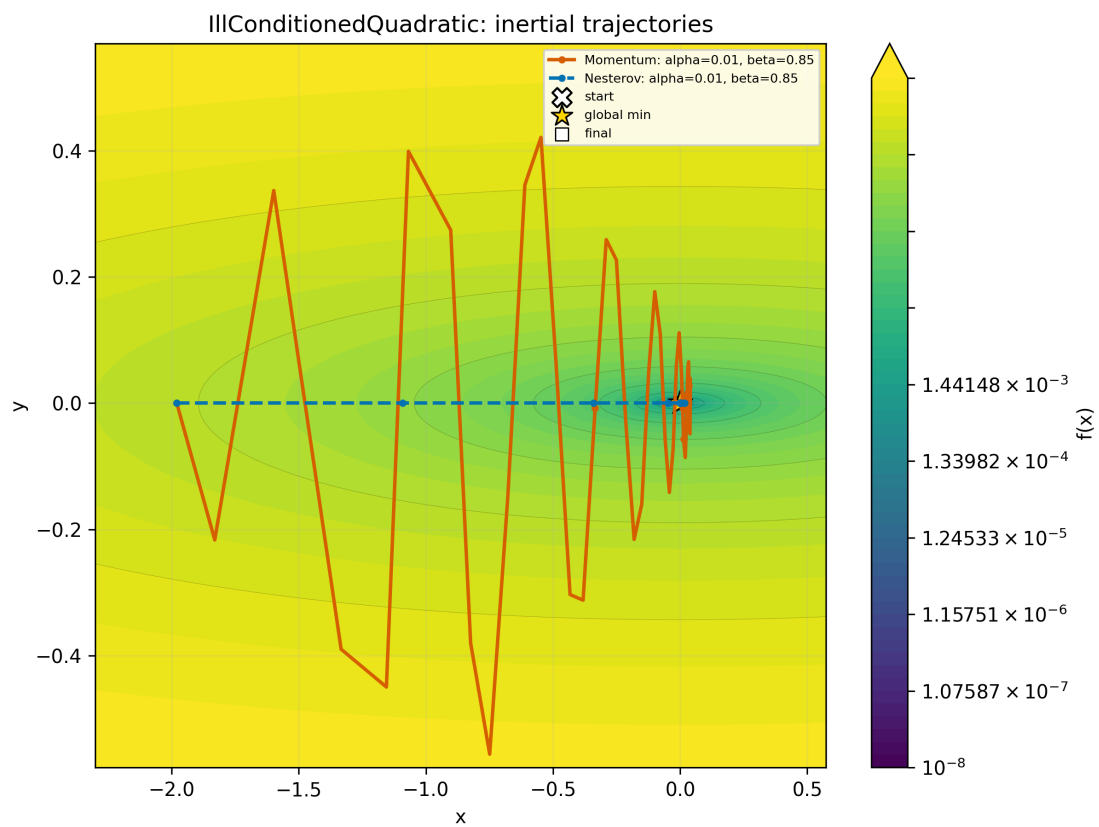


Рис. 15: Инерционные методы на плохо обусловленной квадратичной функции

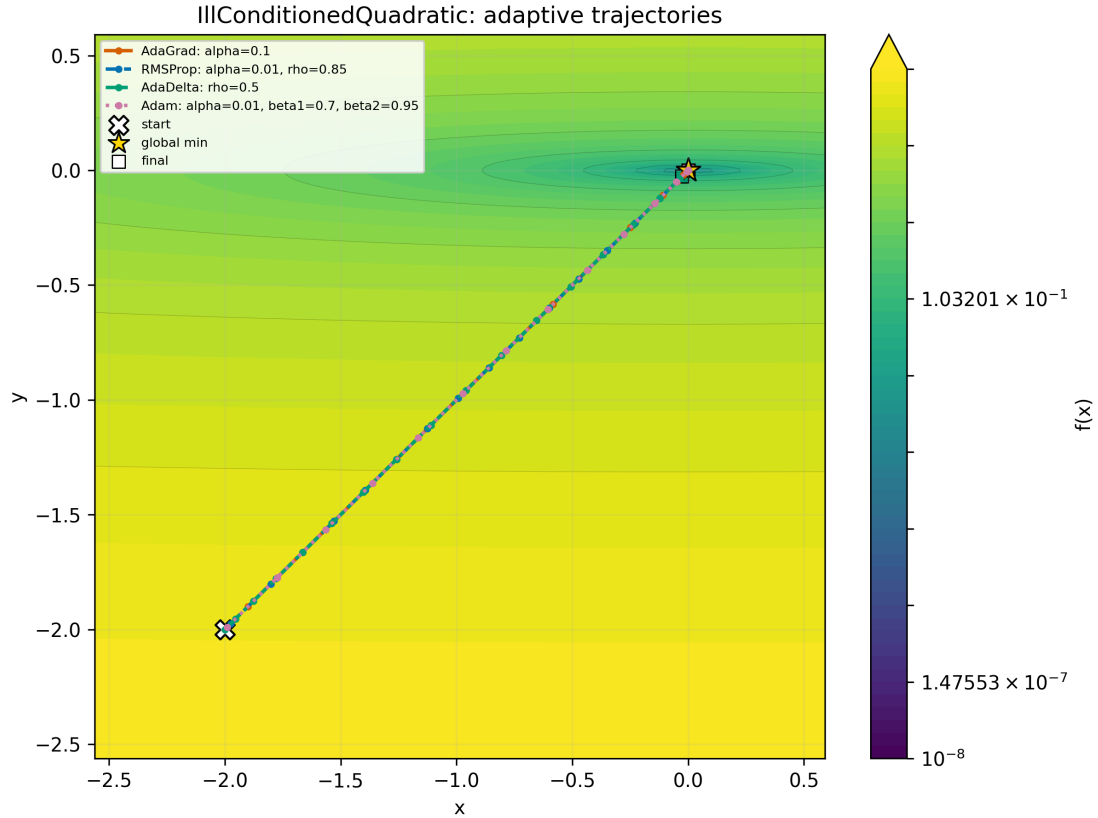


Рис. 16: Адаптивные методы на плохо обусловленной квадратичной функции

Комментарий к графикам. На графиках крестом отмечена стартовая точка, звездой — глобальный минимум, квадратом — конечная точка траектории. Разделение на две группы показывает, что Momentum и Nesterov меняют форму пути через накопленную скорость, а AdaGrad, RMSProp, AdaDelta и Adam — через координатный масштаб обновления.

5.4 Сложные функции: таблицы запусков

Для сложных функций использовались стартовые точки:

- Rosenbrock: $[-1.2, 1.0]$, $[-2.0, 2.0]$, $[0.5, 0.5]$;
- Ackley: $[1.0, 1.0]$, $[-2.0, -2.0]$, $[2.0, -2.0]$;
- Himmelblau: $[2.0, 2.0]$, $[-2.0, -2.0]$, $[4.0, 4.0]$.

Для каждого метода в таблицы включены два явно выбранных набора параметров из сетки. Если метод не выполнил критерий по градиенту, в колонке `converged` указана причина: `max_iter reached`, `diverged`, `overflow` или `nan`.

Таблица 14: Запуски на функции Rosenbrock, $x_0 = [-1.2, 1.0]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Rosenbrock	[-1.2, 1.0]	Momentum	alpha=0.001, beta=0.85	false (max_iter reached)	1500	1501	1501	4.094e-05
Rosenbrock	[-1.2, 1.0]	Momentum	alpha=0.003, beta=0.9	true	1277	1278	1278	1.242e-16
Rosenbrock	[-1.2, 1.0]	Nesterov	alpha=0.001, beta=0.85	false (max_iter reached)	1500	1501	3001	4.633e-05
Rosenbrock	[-1.2, 1.0]	Nesterov	alpha=0.003, beta=0.9	diverged	145	146	292	1.677e+20
Rosenbrock	[-1.2, 1.0]	AdaGrad	alpha=0.01	false (max_iter reached)	1500	1501	1501	4.16631
Rosenbrock	[-1.2, 1.0]	AdaGrad	alpha=0.1	false (max_iter reached)	1500	1501	1501	0.664922
Rosenbrock	[-1.2, 1.0]	RMSPProp	alpha=0.001, rho=0.9	false (max_iter reached)	1500	1501	1501	1.02317
Rosenbrock	[-1.2, 1.0]	RMSPProp	alpha=0.003, rho=0.9	false (max_iter reached)	1500	1501	1501	0.0129424
Rosenbrock	[-1.2, 1.0]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	0.0676546
Rosenbrock	[-1.2, 1.0]	AdaDelta	rho=0.9	false (max_iter reached)	1500	1501	1501	2.05077
Rosenbrock	[-1.2, 1.0]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	false (max_iter reached)	1500	1501	1501	2.32572
Rosenbrock	[-1.2, 1.0]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	false (max_iter reached)	1500	1501	1501	0.0142972

Таблица 15: Запуски на функции Rosenbrock, $x_0 = [-2.0, 2.0]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Rosenbrock	[-2.0, 2.0]	Momentum	alpha=0.001, beta=0.85	false (max_iter reached)	1500	1501	1501	2.598e-05
Rosenbrock	[-2.0, 2.0]	Momentum	alpha=0.003, beta=0.9	diverged	4	5	5	6.313e+36
Rosenbrock	[-2.0, 2.0]	Nesterov	alpha=0.001, beta=0.85	diverged	5	6	12	3.794e+44
Rosenbrock	[-2.0, 2.0]	Nesterov	alpha=0.003, beta=0.9	diverged	3	4	8	7.006e+36
Rosenbrock	[-2.0, 2.0]	AdaGrad	alpha=0.01	false (max_iter reached)	1500	1501	1501	7.39645
Rosenbrock	[-2.0, 2.0]	AdaGrad	alpha=0.1	false (max_iter reached)	1500	1501	1501	6.1811
Rosenbrock	[-2.0, 2.0]	RMSPProp	alpha=0.001, rho=0.9	false (max_iter reached)	1500	1501	1501	4.83695
Rosenbrock	[-2.0, 2.0]	RMSPProp	alpha=0.003, rho=0.9	false (max_iter reached)	1500	1501	1501	0.0933704
Rosenbrock	[-2.0, 2.0]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	4.07624
Rosenbrock	[-2.0, 2.0]	AdaDelta	rho=0.9	false (max_iter reached)	1500	1501	1501	6.19374
Rosenbrock	[-2.0, 2.0]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	false (max_iter reached)	1500	1501	1501	6.18794
Rosenbrock	[-2.0, 2.0]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	false (max_iter reached)	1500	1501	1501	3.86055

Таблица 16: Запуски на функции Rosenbrock, $x_0 = [0.5, 0.5]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Rosenbrock	[0.5, 0.5]	Momentum	alpha=0.001, beta=0.85	false (max_iter reached)	1500	1501	1501	1.875e-05
Rosenbrock	[0.5, 0.5]	Momentum	alpha=0.003, beta=0.9	true	1253	1254	1254	1.228e-16
Rosenbrock	[0.5, 0.5]	Nesterov	alpha=0.001, beta=0.85	false (max_iter reached)	1500	1501	3001	1.610e-05
Rosenbrock	[0.5, 0.5]	Nesterov	alpha=0.003, beta=0.9	diverged	76	77	154	2.748e+49
Rosenbrock	[0.5, 0.5]	AdaGrad	alpha=0.01	false (max_iter reached)	1500	1501	1501	0.127559
Rosenbrock	[0.5, 0.5]	AdaGrad	alpha=0.1	false (max_iter reached)	1500	1501	1501	0.00594364
Rosenbrock	[0.5, 0.5]	RMSPProp	alpha=0.001, rho=0.9	false (max_iter reached)	1500	1501	1501	0.00276421
Rosenbrock	[0.5, 0.5]	RMSPProp	alpha=0.003, rho=0.9	false (max_iter reached)	1500	1501	1501	0.0037705
Rosenbrock	[0.5, 0.5]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	0.0026743
Rosenbrock	[0.5, 0.5]	AdaDelta	rho=0.9	false (max_iter reached)	1500	1501	1501	0.010086
Rosenbrock	[0.5, 0.5]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	false (max_iter reached)	1500	1501	1501	0.00256002
Rosenbrock	[0.5, 0.5]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	false (max_iter reached)	1500	1501	1501	1.530e-06

Таблица 17: Запуски на функции Ackley, $x_0 = [-2.0, -2.0]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Ackley	[-2.0, -2.0]	Momentum	alpha=0.001, beta=0.85	true	165	166	166	6.55965
Ackley	[-2.0, -2.0]	Momentum	alpha=0.003, beta=0.9	true	326	327	327	6.55965
Ackley	[-2.0, -2.0]	Nesterov	alpha=0.001, beta=0.85	true	157	158	315	6.55965
Ackley	[-2.0, -2.0]	Nesterov	alpha=0.003, beta=0.9	true	115	116	231	6.55965
Ackley	[-2.0, -2.0]	AdaGrad	alpha=0.01	true	55	56	56	6.55965
Ackley	[-2.0, -2.0]	AdaGrad	alpha=0.1	true	35	36	36	6.55965
Ackley	[-2.0, -2.0]	RMSPProp	alpha=0.001, rho=0.9	true	60	61	61	6.55965
Ackley	[-2.0, -2.0]	RMSPProp	alpha=0.003, rho=0.9	true	24	25	25	6.55965
Ackley	[-2.0, -2.0]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	6.55982
Ackley	[-2.0, -2.0]	AdaDelta	rho=0.9	true	177	178	178	6.55965
Ackley	[-2.0, -2.0]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	true	312	313	313	6.55965
Ackley	[-2.0, -2.0]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	true	194	195	195	6.55965

Таблица 18: Запуски на функции Ackley, $x_0 = [1.0, 1.0]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Ackley	[1.0, 1.0]	Momentum	alpha=0.001, beta=0.85	true	211	212	212	3.57445
Ackley	[1.0, 1.0]	Momentum	alpha=0.003, beta=0.9	true	323	324	324	3.57445
Ackley	[1.0, 1.0]	Nesterov	alpha=0.001, beta=0.85	true	160	161	321	3.57445
Ackley	[1.0, 1.0]	Nesterov	alpha=0.003, beta=0.9	true	125	126	251	3.57445
Ackley	[1.0, 1.0]	AdaGrad	alpha=0.01	true	81	82	82	3.57445
Ackley	[1.0, 1.0]	AdaGrad	alpha=0.1	true	36	37	37	3.57445
Ackley	[1.0, 1.0]	RMSProp	alpha=0.001, rho=0.9	true	69	70	70	3.57445
Ackley	[1.0, 1.0]	RMSProp	alpha=0.003, rho=0.9	true	30	31	31	3.57445
Ackley	[1.0, 1.0]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	3.57462
Ackley	[1.0, 1.0]	AdaDelta	rho=0.9	true	202	203	203	3.57445
Ackley	[1.0, 1.0]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	true	336	337	337	3.57445
Ackley	[1.0, 1.0]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	true	217	218	218	3.57445

Таблица 19: Запуски на функции Ackley, $x_0 = [2.0, -2.0]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Ackley	[2.0, -2.0]	Momentum	alpha=0.001, beta=0.85	true	165	166	166	6.55965
Ackley	[2.0, -2.0]	Momentum	alpha=0.003, beta=0.9	true	326	327	327	6.55965
Ackley	[2.0, -2.0]	Nesterov	alpha=0.001, beta=0.85	true	157	158	315	6.55965
Ackley	[2.0, -2.0]	Nesterov	alpha=0.003, beta=0.9	true	115	116	231	6.55965
Ackley	[2.0, -2.0]	AdaGrad	alpha=0.01	true	55	56	56	6.55965
Ackley	[2.0, -2.0]	AdaGrad	alpha=0.1	true	35	36	36	6.55965
Ackley	[2.0, -2.0]	RMSProp	alpha=0.001, rho=0.9	true	60	61	61	6.55965
Ackley	[2.0, -2.0]	RMSProp	alpha=0.003, rho=0.9	true	24	25	25	6.55965
Ackley	[2.0, -2.0]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	6.55982
Ackley	[2.0, -2.0]	AdaDelta	rho=0.9	true	177	178	178	6.55965
Ackley	[2.0, -2.0]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	true	312	313	313	6.55965
Ackley	[2.0, -2.0]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	true	194	195	195	6.55965

Таблица 20: Запуски на функции Himmelblau, $x_0 = [-2.0, -2.0]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Himmelblau	[-2.0, -2.0]	Momentum	alpha=0.001, beta=0.85	true	259	260	260	3.123e-19
Himmelblau	[-2.0, -2.0]	Momentum	alpha=0.003, beta=0.9	true	422	423	423	3.981e-19
Himmelblau	[-2.0, -2.0]	Nesterov	alpha=0.001, beta=0.85	true	170	171	341	5.320e-19
Himmelblau	[-2.0, -2.0]	Nesterov	alpha=0.003, beta=0.9	true	124	125	249	3.800e-19
Himmelblau	[-2.0, -2.0]	AdaGrad	alpha=0.01	false (max_iter reached)	1500	1501	1501	42.9012
Himmelblau	[-2.0, -2.0]	AdaGrad	alpha=0.1	true	1095	1096	1096	4.883e-19
Himmelblau	[-2.0, -2.0]	RMSProp	alpha=0.001, rho=0.9	false (max_iter reached)	1500	1501	1501	3.82405
Himmelblau	[-2.0, -2.0]	RMSProp	alpha=0.003, rho=0.9	false (max_iter reached)	1500	1501	1501	2.936e-04
Himmelblau	[-2.0, -2.0]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	6.389e-05
Himmelblau	[-2.0, -2.0]	AdaDelta	rho=0.9	false (max_iter reached)	1500	1501	1501	12.1448
Himmelblau	[-2.0, -2.0]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	false (max_iter reached)	1500	1501	1501	5.29379
Himmelblau	[-2.0, -2.0]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	true	1058	1059	1059	3.573e-19

Таблица 21: Запуски на функции Himmelblau, $x_0 = [2.0, 2.0]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Himmelblau	[2.0, 2.0]	Momentum	alpha=0.001, beta=0.85	true	258	259	259	1.212e-18
Himmelblau	[2.0, 2.0]	Momentum	alpha=0.003, beta=0.9	true	402	403	403	3.364e-19
Himmelblau	[2.0, 2.0]	Nesterov	alpha=0.001, beta=0.85	true	198	199	397	1.663e-18
Himmelblau	[2.0, 2.0]	Nesterov	alpha=0.003, beta=0.9	true	212	213	425	1.476e-18
Himmelblau	[2.0, 2.0]	AdaGrad	alpha=0.01	false (max_iter reached)	1500	1501	1501	3.42278
Himmelblau	[2.0, 2.0]	AdaGrad	alpha=0.1	true	550	551	551	9.291e-19
Himmelblau	[2.0, 2.0]	RMSProp	alpha=0.001, rho=0.9	false (max_iter reached)	1500	1501	1501	1.849e-05
Himmelblau	[2.0, 2.0]	RMSProp	alpha=0.003, rho=0.9	false (max_iter reached)	1500	1501	1501	1.667e-04
Himmelblau	[2.0, 2.0]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	1.016e-04
Himmelblau	[2.0, 2.0]	AdaDelta	rho=0.9	false (max_iter reached)	1500	1501	1501	0.2069
Himmelblau	[2.0, 2.0]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	false (max_iter reached)	1500	1501	1501	8.443e-13
Himmelblau	[2.0, 2.0]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	true	796	797	797	4.802e-19

Таблица 22: Запуски на функции Himmelblau, $x_0 = [4.0, 4.0]$

function	x0	optimizer	params	converged	iterations	f_calls	grad_calls	$f(x_k)$
Himmelblau	[4.0, 4.0]	Momentum	alpha=0.001, beta=0.85	true	286	287	287	9.206e-19
Himmelblau	[4.0, 4.0]	Momentum	alpha=0.003, beta=0.9	true	425	426	426	5.071e-19
Himmelblau	[4.0, 4.0]	Nesterov	alpha=0.001, beta=0.85	true	214	215	429	6.788e-19
Himmelblau	[4.0, 4.0]	Nesterov	alpha=0.003, beta=0.9	true	211	212	423	6.249e-19
Himmelblau	[4.0, 4.0]	AdaGrad	alpha=0.01	false (max_iter reached)	1500	1501	1501	69.5884
Himmelblau	[4.0, 4.0]	AdaGrad	alpha=0.1	false (max_iter reached)	1500	1501	1501	0.00102637
Himmelblau	[4.0, 4.0]	RMSProp	alpha=0.001, rho=0.9	false (max_iter reached)	1500	1501	1501	4.80355
Himmelblau	[4.0, 4.0]	RMSProp	alpha=0.003, rho=0.9	false (max_iter reached)	1500	1501	1501	1.663e-04
Himmelblau	[4.0, 4.0]	AdaDelta	rho=0.5	false (max_iter reached)	1500	1501	1501	0.380447
Himmelblau	[4.0, 4.0]	AdaDelta	rho=0.9	false (max_iter reached)	1500	1501	1501	28.2961
Himmelblau	[4.0, 4.0]	Adam	alpha=0.001, beta1=0.9, beta2=0.99	false (max_iter reached)	1500	1501	1501	8.72992
Himmelblau	[4.0, 4.0]	Adam	alpha=0.003, beta1=0.85, beta2=0.99	true	1433	1434	1434	1.211e-18

Комментарий к таблицам. Сложные функции нельзя оценивать только по одному старту. На Rosenbrock методы часто доходят до лимита из-за узкой изогнутой долины. На Himmelblau сходимость зависит от области притяжения. На Ackley возможна остановка около локальной особенности поверхности: критерий по градиенту может выполняться, хотя точка не совпадает с глобальным минимумом $(0, 0)$.

5.5 Сложные функции: траектории

Для траекторий на сложных функциях снова выбран один представительный набор параметров на метод по правилу: лучший сошедшийся запуск по числу итераций, иначе лучший конечный результат по значению функции. Для Ackley окно построения расширено так, чтобы был виден глобальный минимум $(0, 0)$, а не только область около стартовой точки.

5.5.1 Rosenbrock

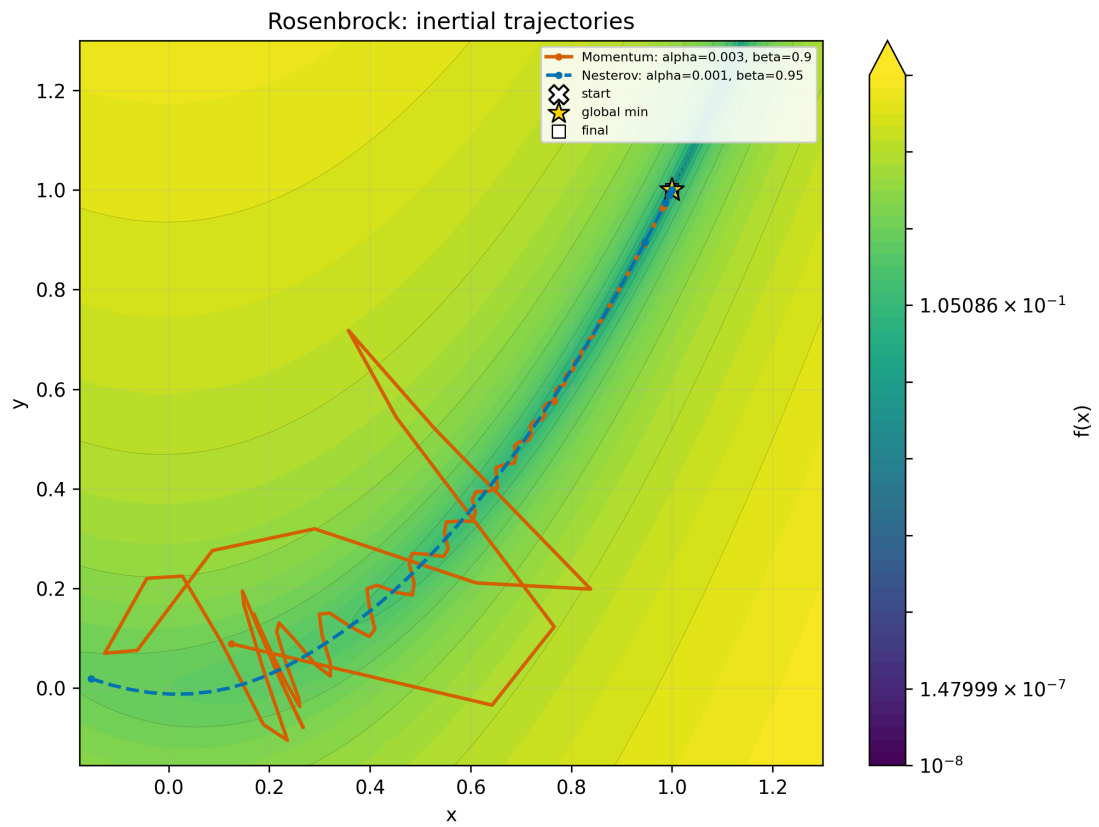


Рис. 17: Rosenbrock, $x_0 = (-1.2, 1.0)$: инерционные методы

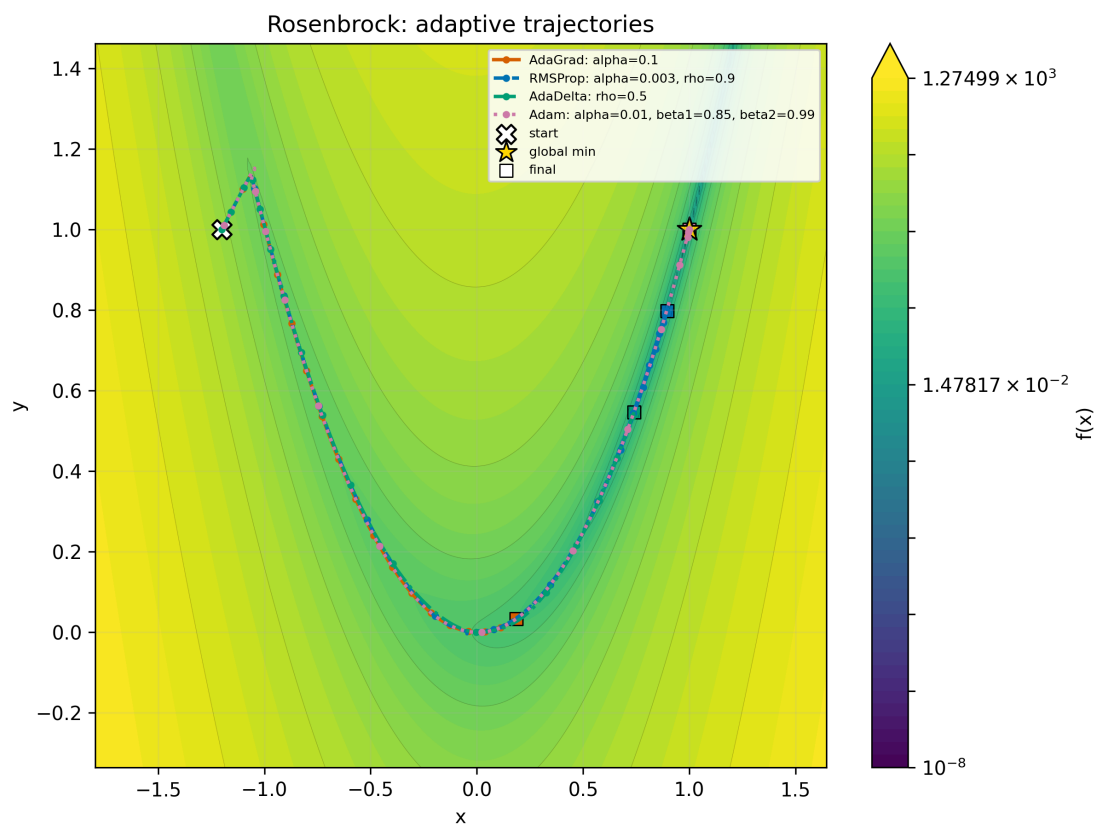


Рис. 18: Rosenbrock, $x_0 = (-1.2, 1.0)$: адаптивные методы

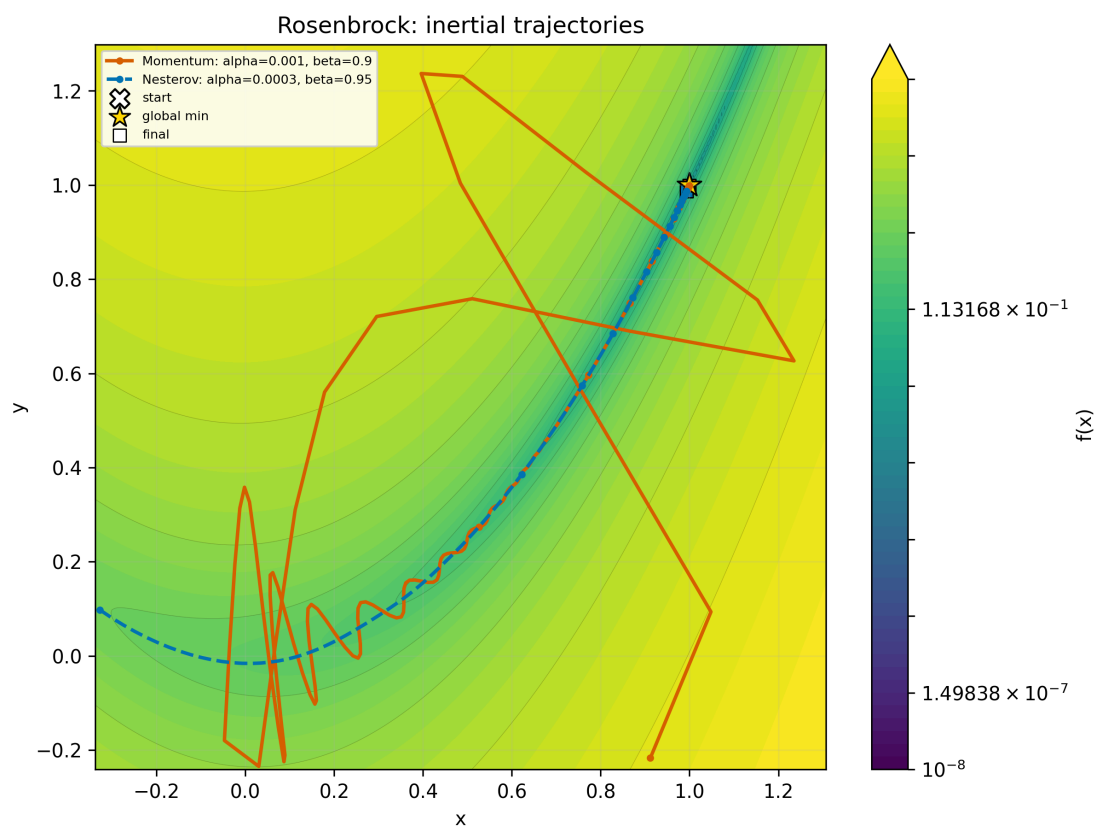


Рис. 19: Rosenbrock, $x_0 = (-2.0, 2.0)$: инерционные методы

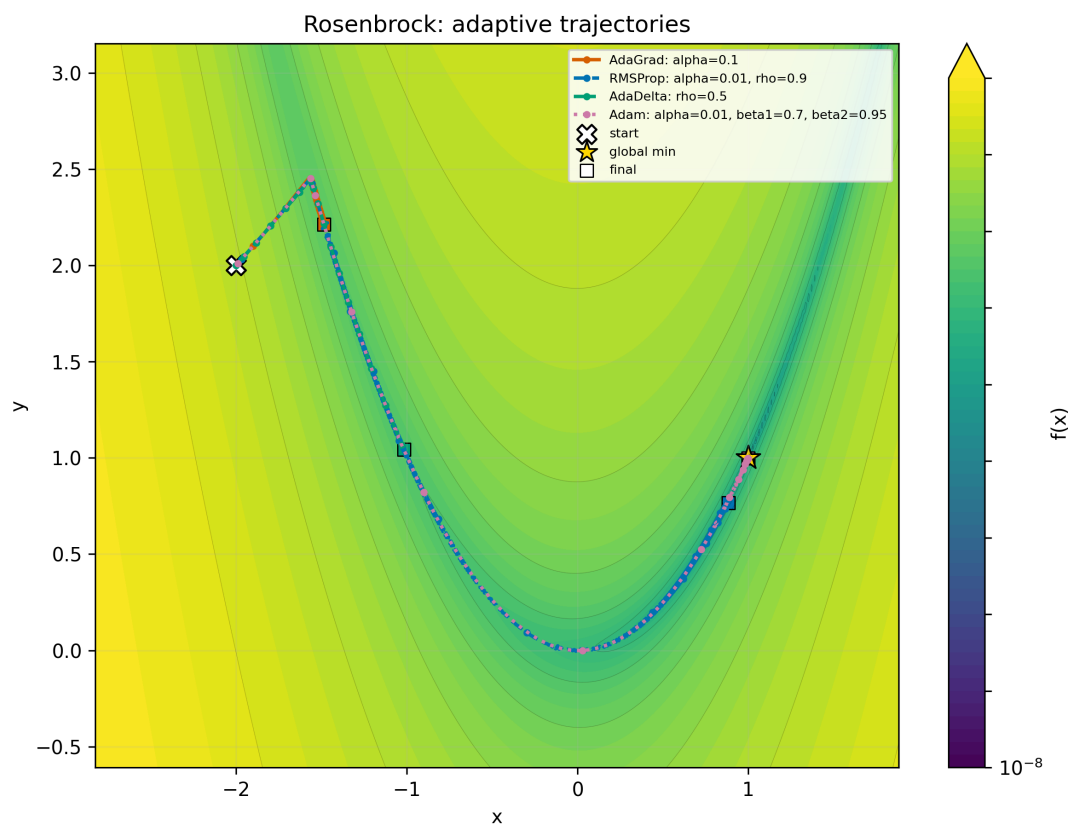


Рис. 20: Rosenbrock, $x_0 = (-2.0, 2.0)$: адаптивные методы

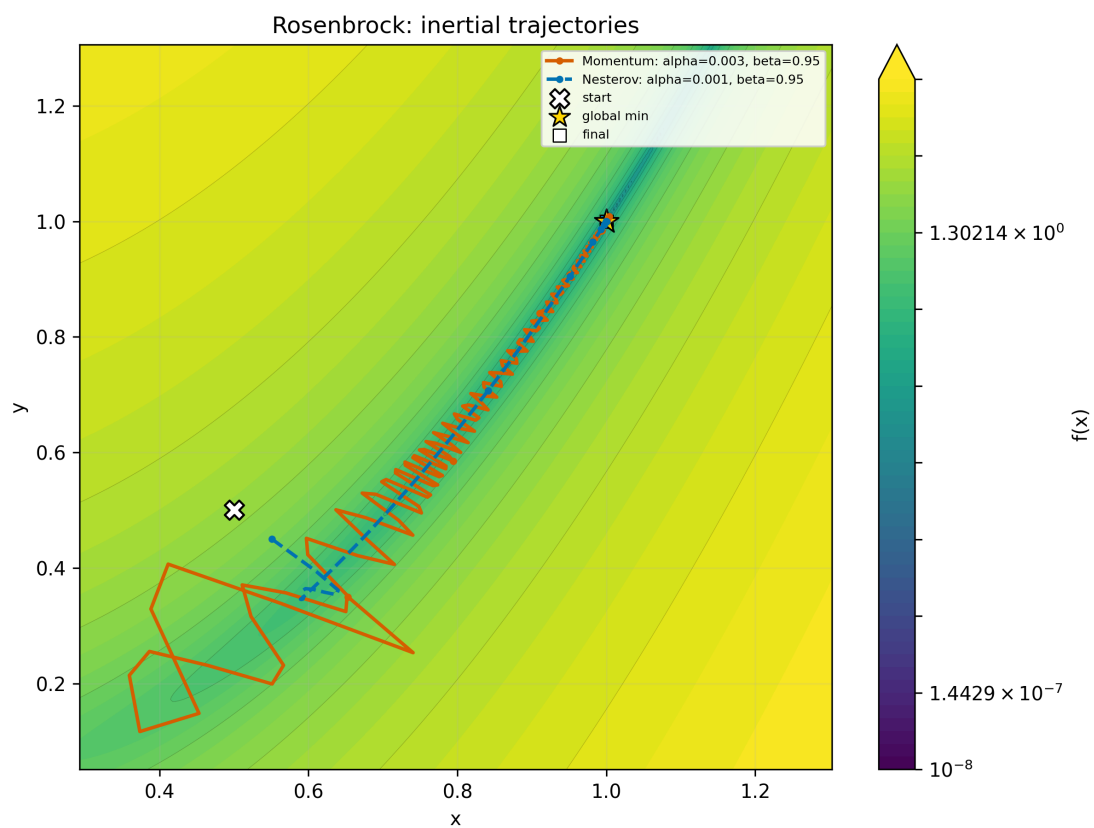


Рис. 21: Rosenbrock, $x_0 = (0.5, 0.5)$: инерционные методы

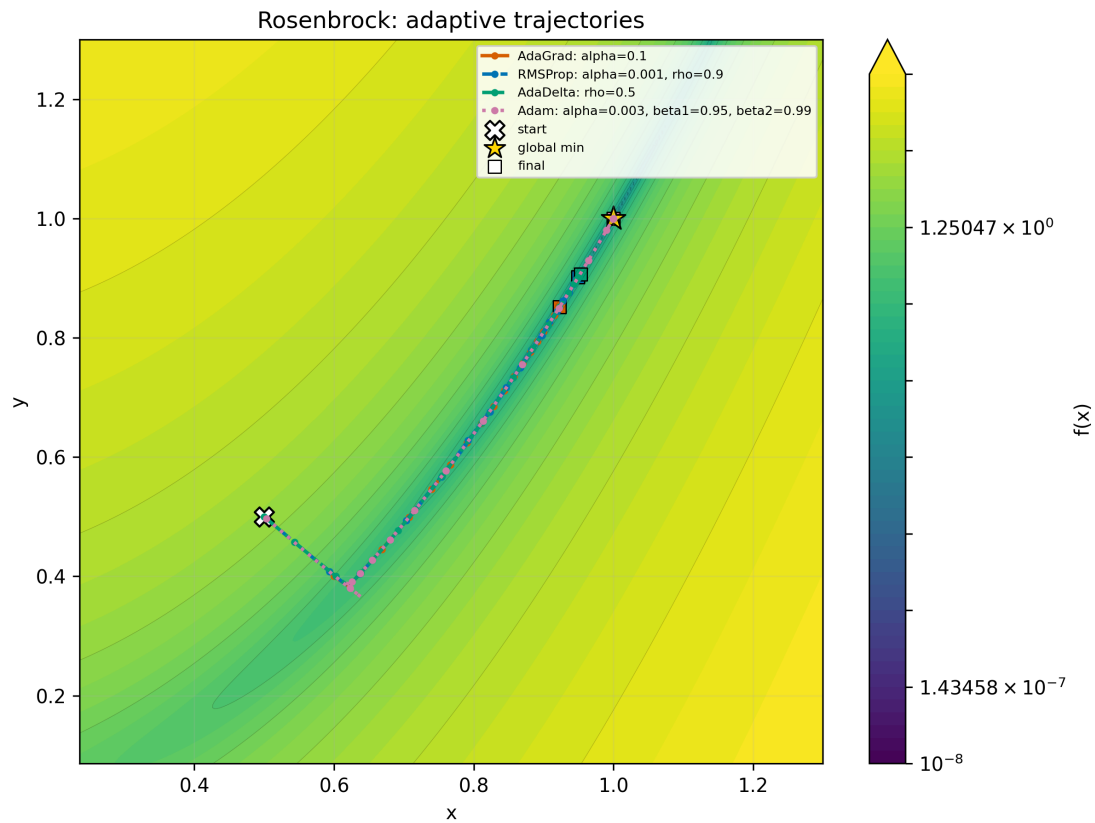


Рис. 22: Rosenbrock, $x_0 = (0.5, 0.5)$: адаптивные методы

Комментарий к графикам. У Rosenbrock узкая кривая долина, поэтому на разных стартах методы могут выглядеть устойчивыми или, наоборот, долго идти вдоль стенки долины. Инерция иногда ускоряет продвижение, но может уводить поперек долины.

5.5.2 Ackley

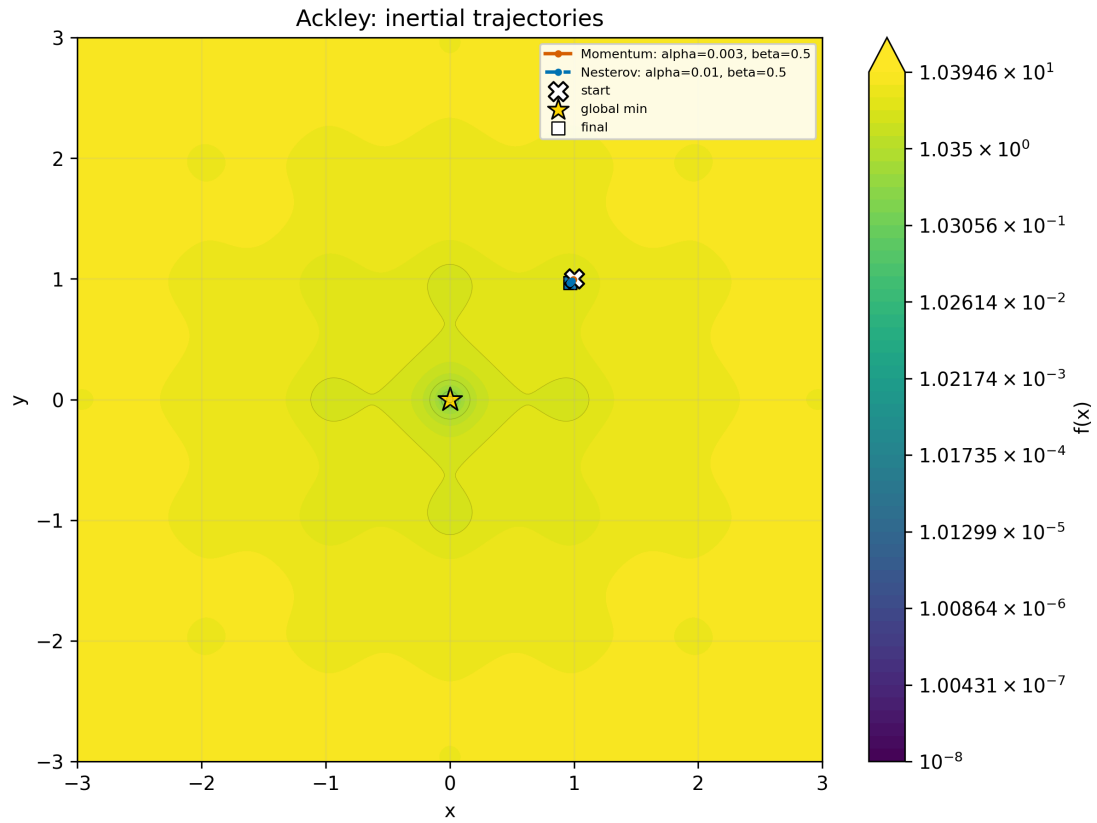


Рис. 23: Ackley, $x_0 = (1.0, 1.0)$: инерционные методы

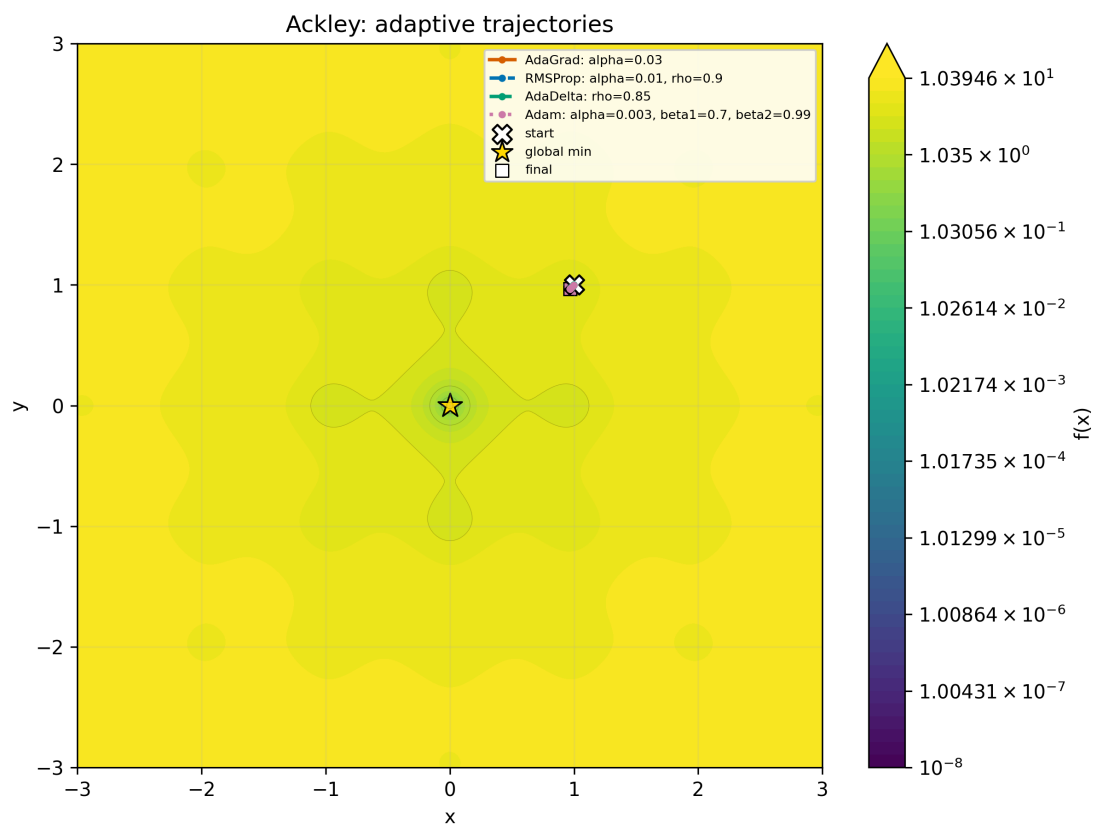


Рис. 24: Ackley, $x_0 = (1.0, 1.0)$: адаптивные методы

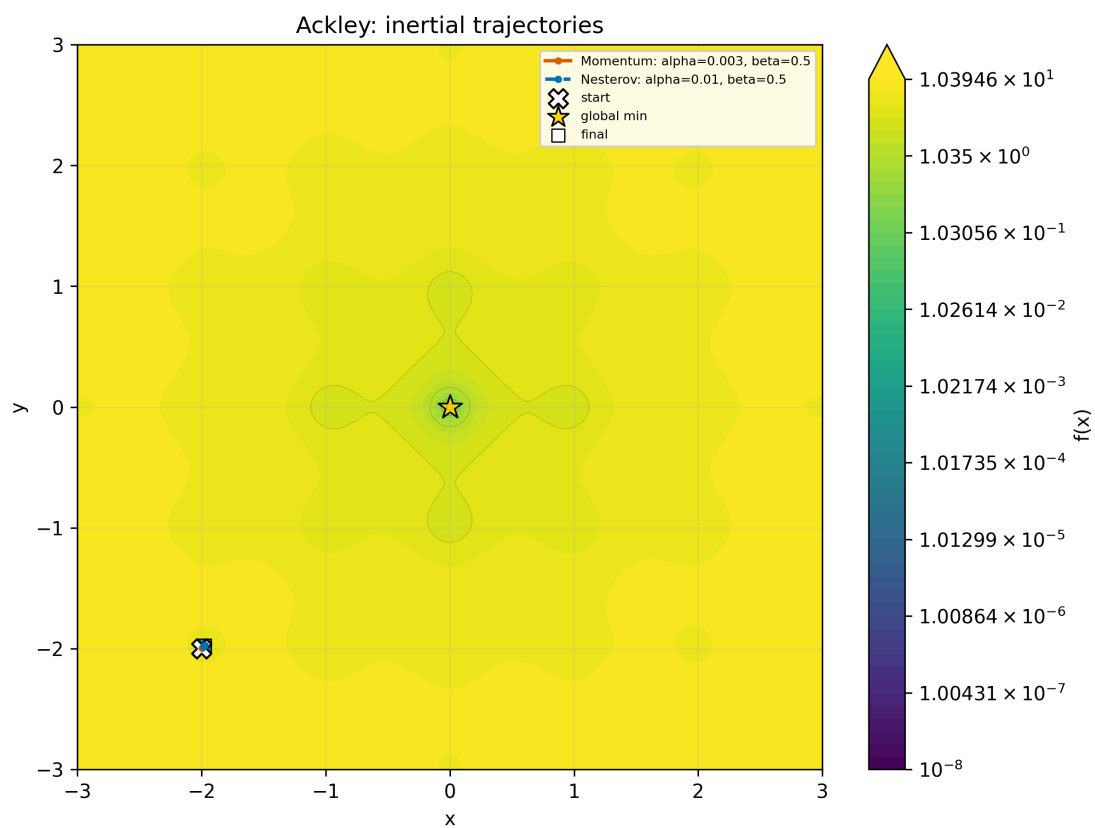


Рис. 25: Ackley, $x_0 = (-2.0, -2.0)$: инерционные методы

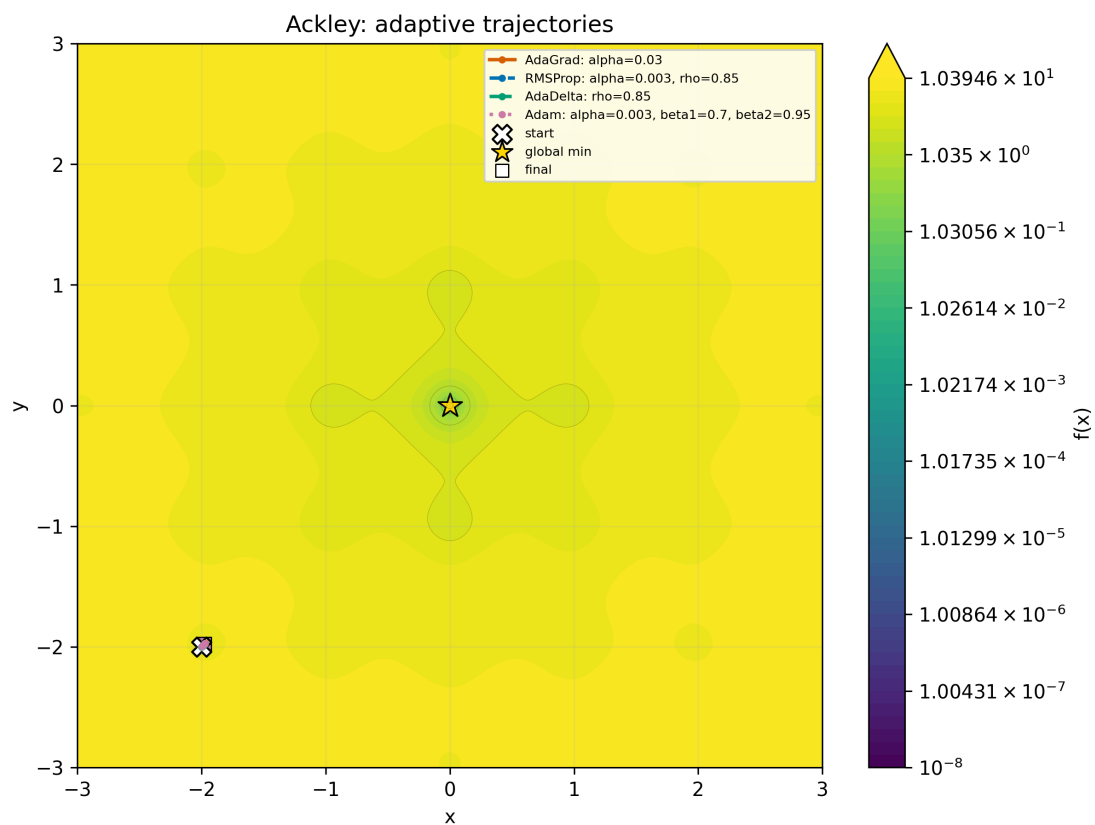


Рис. 26: Ackley, $x_0 = (-2.0, -2.0)$: адаптивные методы

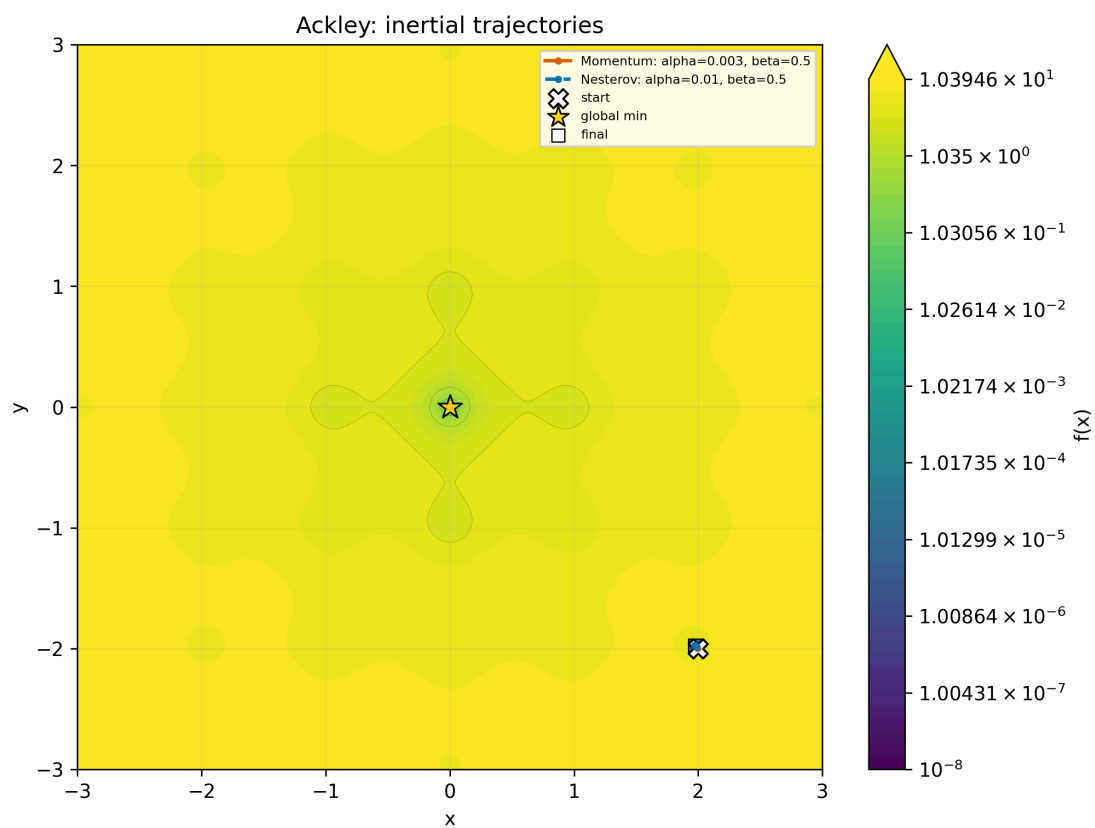


Рис. 27: Ackley, $x_0 = (2.0, -2.0)$: инерционные методы

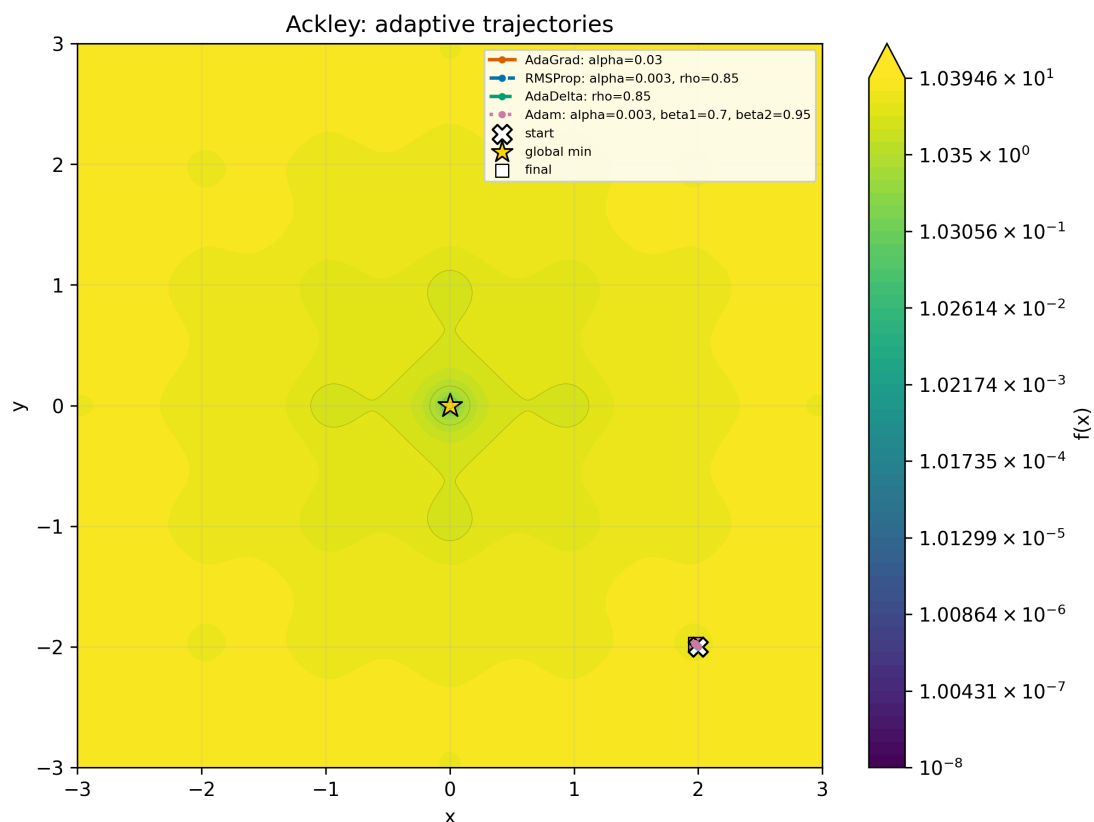


Рис. 28: Ackley, $x_0 = (2.0, -2.0)$: адаптивные методы

Комментарий к графикам. Функция Экли имеет глобальный минимум в $(0, 0)$, но ее поверхность содержит локальные колебания. Если конечная точка отмечена не около звезды минимума, это означает, что метод остановился или застрял в другой области; поэтому для Ackley особенно важно смотреть не только на `converged`, но и на $f(x_k)$ и положение конечной точки.

5.5.3 Himmelblau

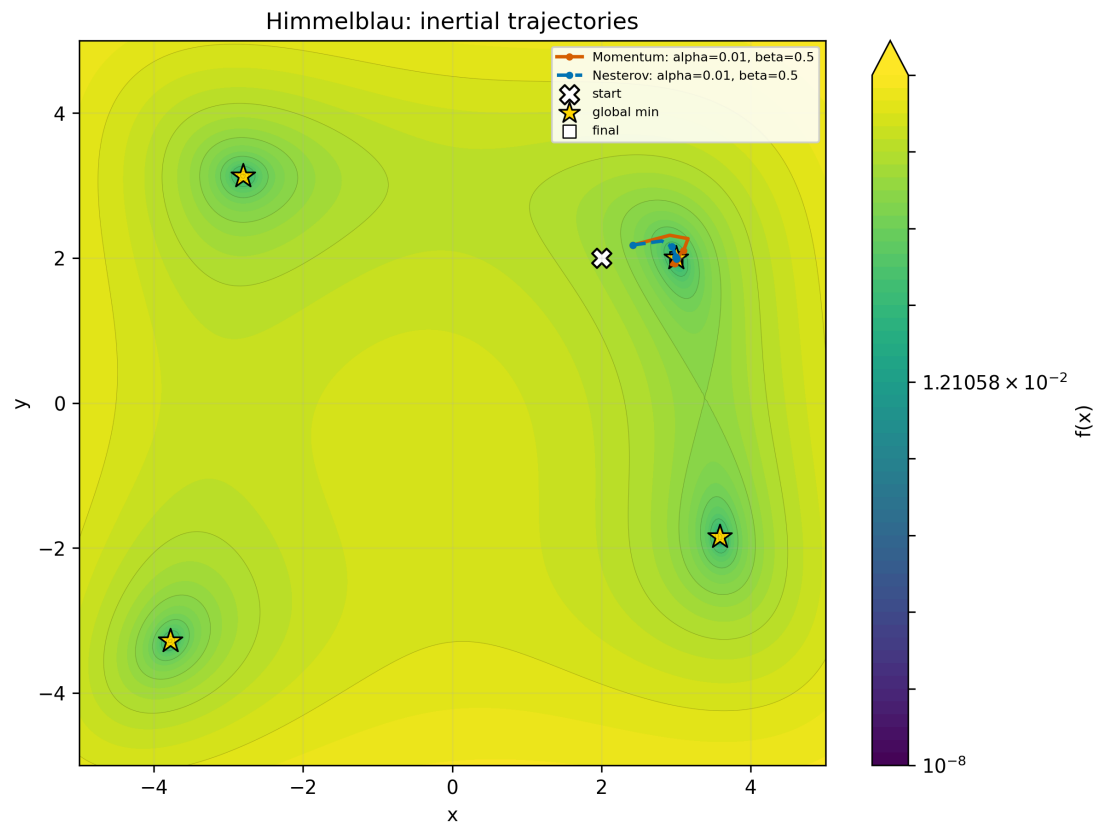


Рис. 29: Himmelblau, $x_0 = (2.0, 2.0)$: инерционные методы

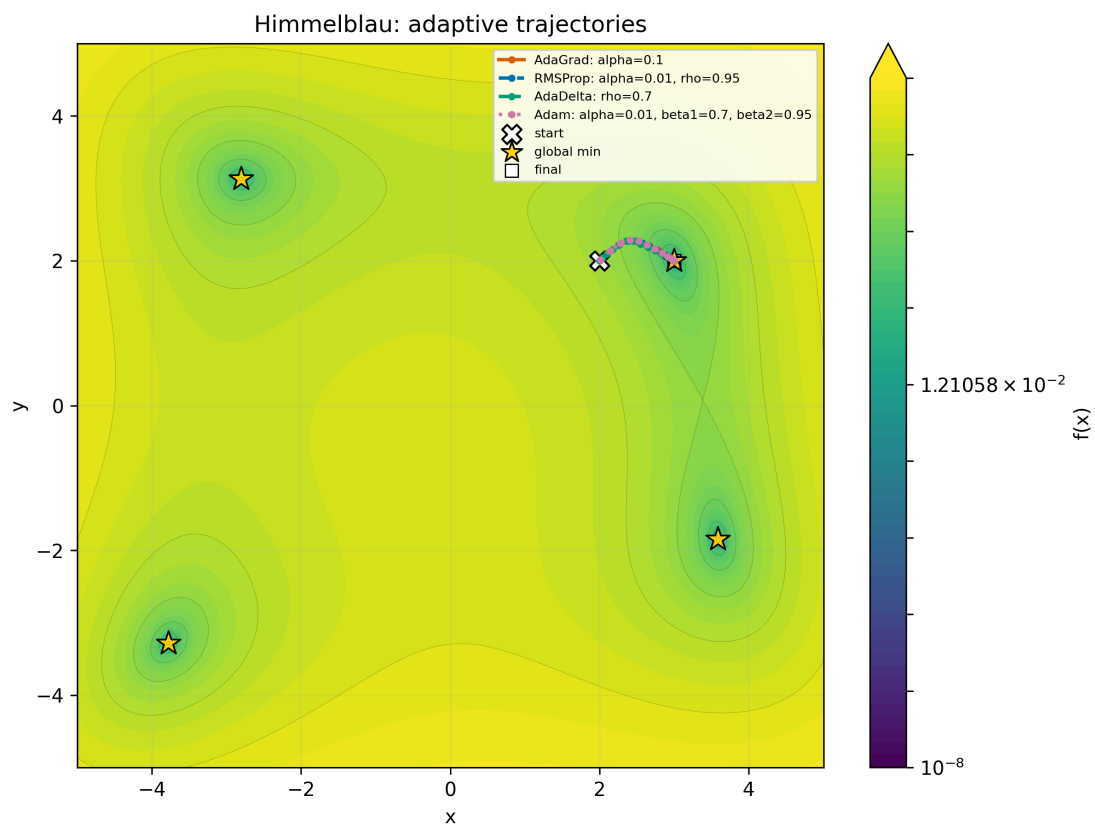


Рис. 30: Himmelblau, $x_0 = (2.0, 2.0)$: адаптивные методы

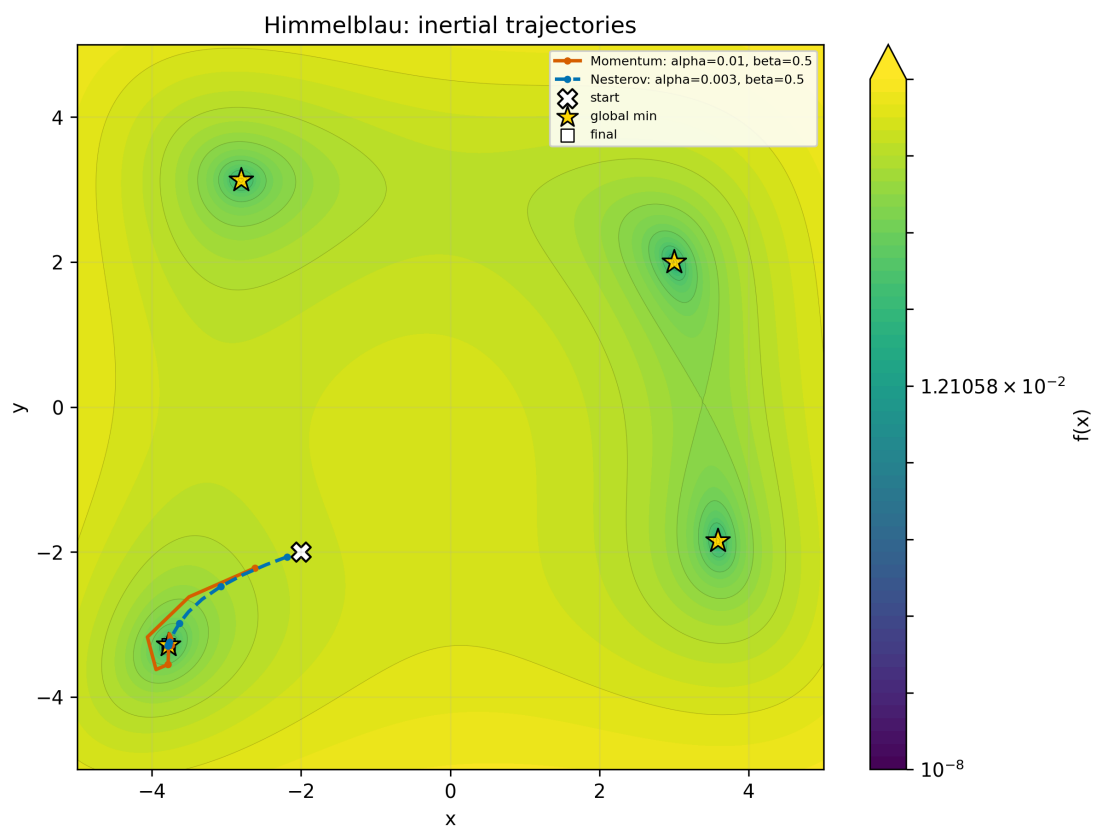


Рис. 31: Himmelblau, $x_0 = (-2.0, -2.0)$: инерционные методы

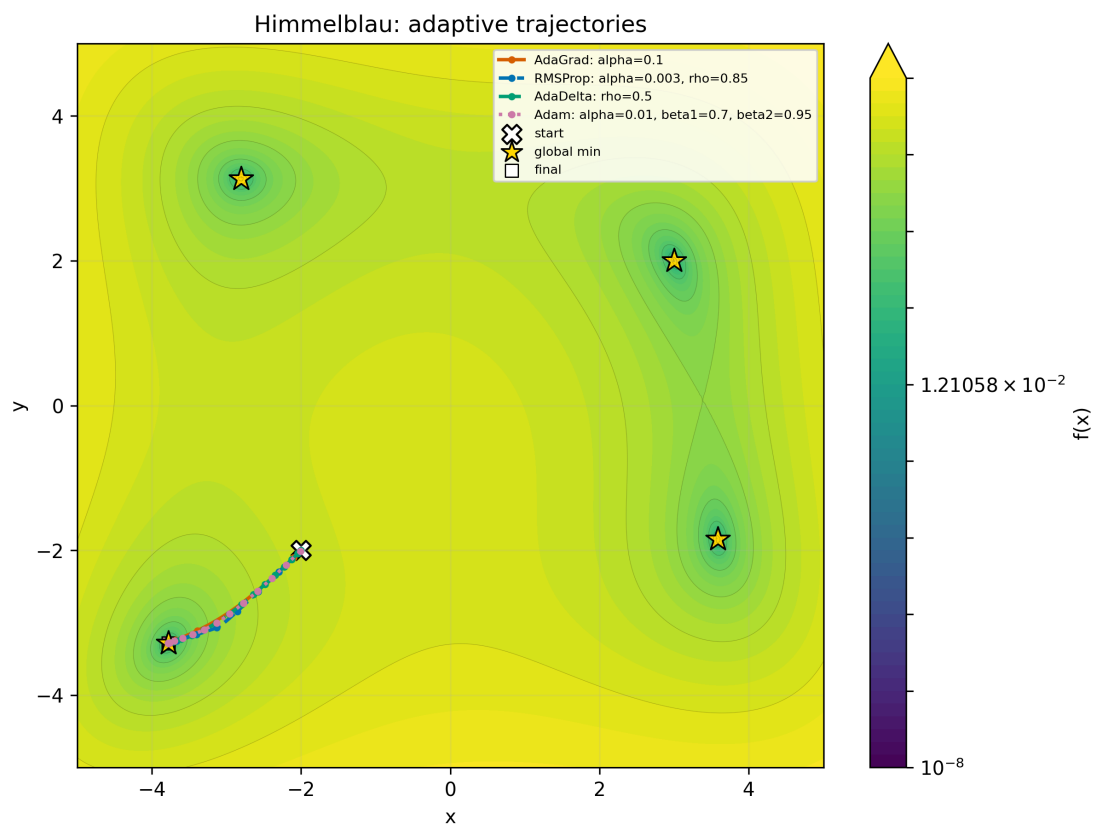


Рис. 32: Himmelblau, $x_0 = (-2.0, -2.0)$: адаптивные методы

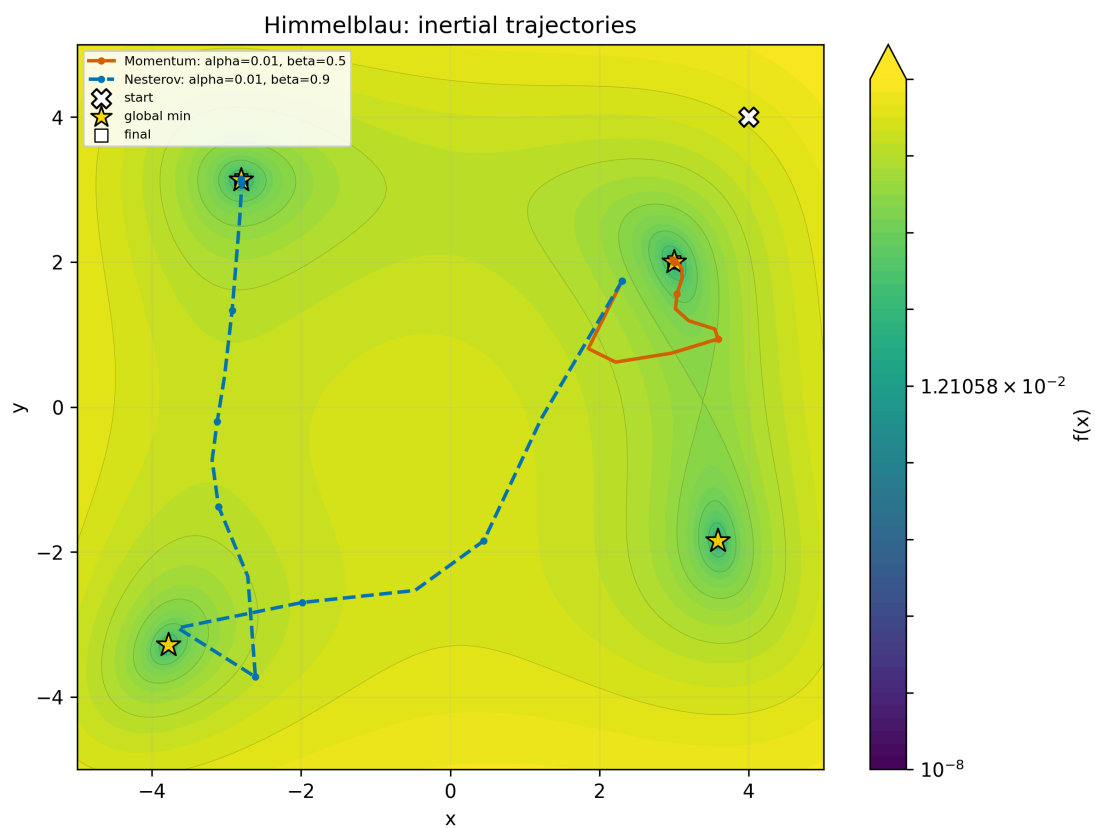


Рис. 33: Himmelblau, $x_0 = (4.0, 4.0)$: инерционные методы

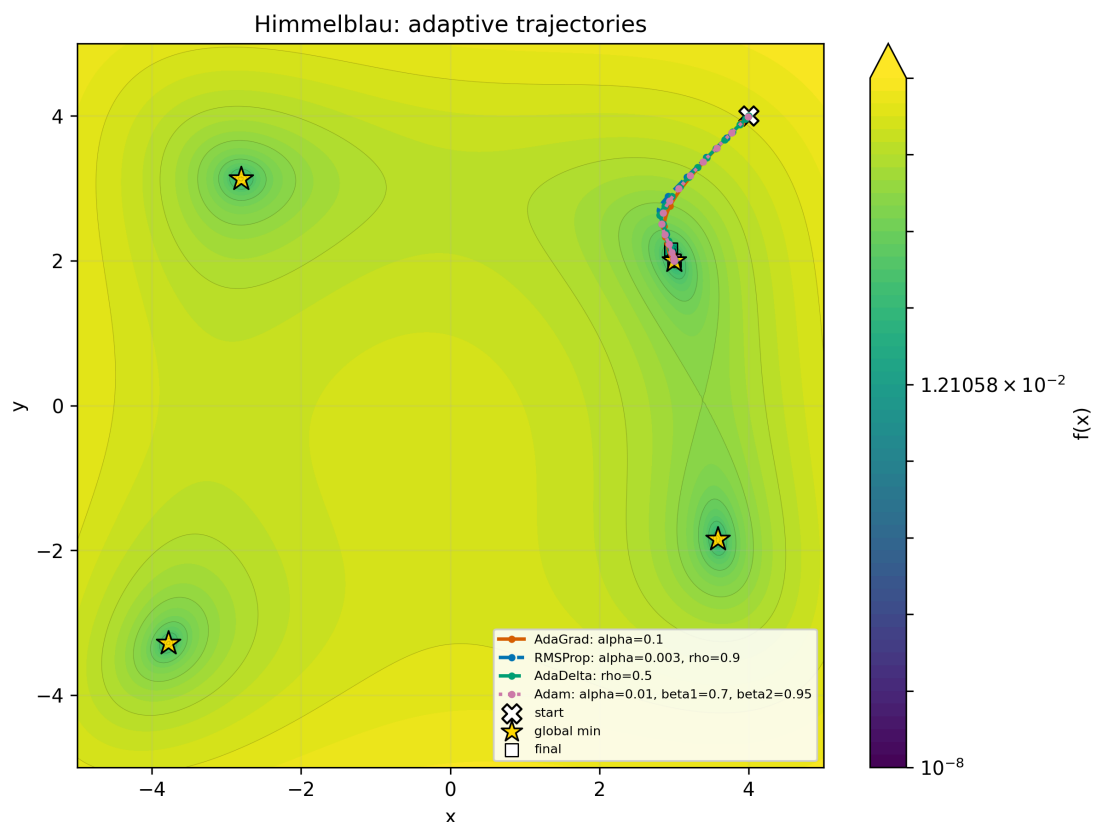


Рис. 34: Himmelblau, $x_0 = (4.0, 4.0)$: адаптивные методы

Комментарий к графикам. У Himmelblau несколько глобальных минимумов, поэтому стартовая точка определяет область притяжения. Конечные точки могут быть разными, но при малом значении функции это соответствует попаданию в один из глобальных минимумов.

6 Сводная таблица эксперимента

Полная сводка всех запусков сохраняется в файле:

`../outputs/lab3/summary.csv`

В основной части отчета оставлены компактные таблицы по ключевым экспериментам, чтобы не перегружать PDF полной CSV-таблицей.

7 ОБЩИЙ ВЫВОД

В работе были реализованы и исследованы шесть модификаций градиентного спуска: Momentum, Nesterov, AdaGrad, RMSProp, AdaDelta и Adam. Эксперименты подтвердили, что инерция и адаптивное масштабирование шага могут

существенно ускорить сходимость, но требуют аккуратного выбора гиперпараметров.

На квадратичных функциях наиболее устойчивыми оказались Momentum, Nesterov, RMSProp и Adam. AdaGrad и AdaDelta в выбранной сетке часто не достигали точности 10^{-8} до лимита итераций, потому что их эффективные шаги становились слишком малыми.

На сложных функциях результат сильнее зависит от стартовой точки.

Rosenbrock остается трудным из-за узкой изогнутой долины. Ackley может приводить методы к остановке около локальных особенностей поверхности. Himmelblau демонстрирует влияние областей притяжения нескольких глобальных минимумов.